

LineStacker documentation

Release 2.0.0

Jean-Baptiste Jolly

Oct 30, 2026

USER GUIDE

1 Installation 3

2 How stacking works 5

3 Stacking positions 7

4 Stacking cubes 9

5 Stacking spectra 13

6 Analysis tools 15

7 Examples 17

8 LineStacker examples 19

9 API reference 33

10 LineStacker 2.0: Python 3 / CASA 6 port 51

11 Citing LineStacker 59

12 Indices 61

Python Module Index 63

Index 65

LineStacker stacks the spectral lines of many faint sources to measure their average emission: the line of each source is centred on its own redshift, then all sources are averaged channel by channel. It stacks

- **data cubes**, FITS files or CASA images, around each source (`LineStacker.image`), for lines or continuum;
- **1D spectra** already extracted (`LineStacker.OneD_Stacker`);

and comes with tools to test what a stack shows: bootstrapping, subsampling to find the sources that dominate it, stacks at random positions, noise estimates, and rebinning or regridding of lines of different widths and redshifts.

```
import LineStacker
import LineStacker.image

# one row per source: ra, dec, z and the cube it lies in
coords = LineStacker.readCatalog('sources.csv')
fEm = 1897420620253.1646          # rest frequency of the line (Hz)
result = LineStacker.image.stack(coords, fEm=fEm, stampsize=16, N_chans=31)
# LineStacker: mean line stack of 30 sources (16x16 pixels, 31 channels)
#              written to .../stackResult.fits
spectrum = result[1]          # stacked spectrum of the central pixel
```

LineStacker 2.0 runs on Python 3 and CASA 6 (CASA is only needed for CASA images: FITS cubes need no CASA). See *what changed in 2.0* if you used LineStacker 1.x.

CHAPTER

ONE

INSTALLATION

LineStacker needs Python 3.9 or newer, NumPy, SciPy and astropy.

```
pip install LineStacker                  # FITS cubes and 1D spectra
pip install "LineStacker[casa]"          # also CASA images (casatools,␣
 ↪casatasks)
pip install "LineStacker[plot]"          # matplotlib, for the plotting␣
 ↪options
```

To install the development version, clone the repository and install it in editable mode:

```
git clone https://github.com/jbjolly/LineStacker.git
cd LineStacker
pip install -e ".[casa,plot]"
```

1.1 CASA images

FITS cubes are read with astropy and need no CASA. To read and write CASA images (`.image` directories), LineStacker uses CASA 6's `casatools`, the modular CASA from PyPI (tested with CASA 6.7.6):

```
pip install casatools
```

`casatools` needs CASA's runtime data (the measures tables). By default it downloads them into `~/.casa/data` the first time it is imported; see External data in the CASA documentation to set another place, for example without internet access:

```
# ~/.casa/config.py
measurespath = "/path/to/casa/data"
measures_auto_update = False
data_auto_update = False
```

1.1.1 From the CASA prompt

LineStacker also runs inside the monolithic CASA (the CASA tarball with its own Python). Install its dependencies with CASA's own `pip`, so they match the NumPy that CASA ships:

```
PYTHONNOUSERSITE=1 /path/to/casa-6.7.6/bin/pip3 install "astropy>=7.2"
PYTHONNOUSERSITE=1 /path/to/casa-6.7.6/bin/pip3 install /path/to/
 ↪LineStacker
```

Mixing CASA with packages from another environment (a virtual environment on `PYTHONPATH`, say) breaks when the versions disagree: CASA 6.7 ships NumPy 2.4, which astropy older than 7.2 cannot use (the error is `module 'numpy' has no attribute 'in1d'`). `import LineStacker` warns when it finds that combination.

1.1.2 CASA's log file

CASA is only started when LineStacker reads or writes a CASA image. When LineStacker starts it, CASA's log goes to a single file in the temporary directory, `<tmp>/linestacker-casa-<user>.log`, and CASA's messages go there rather than to the terminal: no new `casa-<date>.log` appears in the working directory at every run. Set `LineStacker.casa_logfile` before the first CASA image is read to change this:

```
import LineStacker
LineStacker.casa_logfile = 'stacking.log'    # this file
LineStacker.casa_logfile = None              # no log file
LineStacker.casa_logfile = 'casa'            # CASA's default, casa-<date>.
 ↪log here
```

A `logfile` set in `~/.casa/config.py` is used as it is. Inside a CASA session, or after importing `casatools` or `casatasks` yourself, CASA's own settings apply.

1.2 Checking the installation

The tests run on synthetic cubes and spectra (the CASA tests are skipped when casatools is not installed):

```
pip install pytest
python -m pytest tests
```

The examples in `LineStacker/Example` run on the example data of the repository; see *Examples*.

1.3 Messages

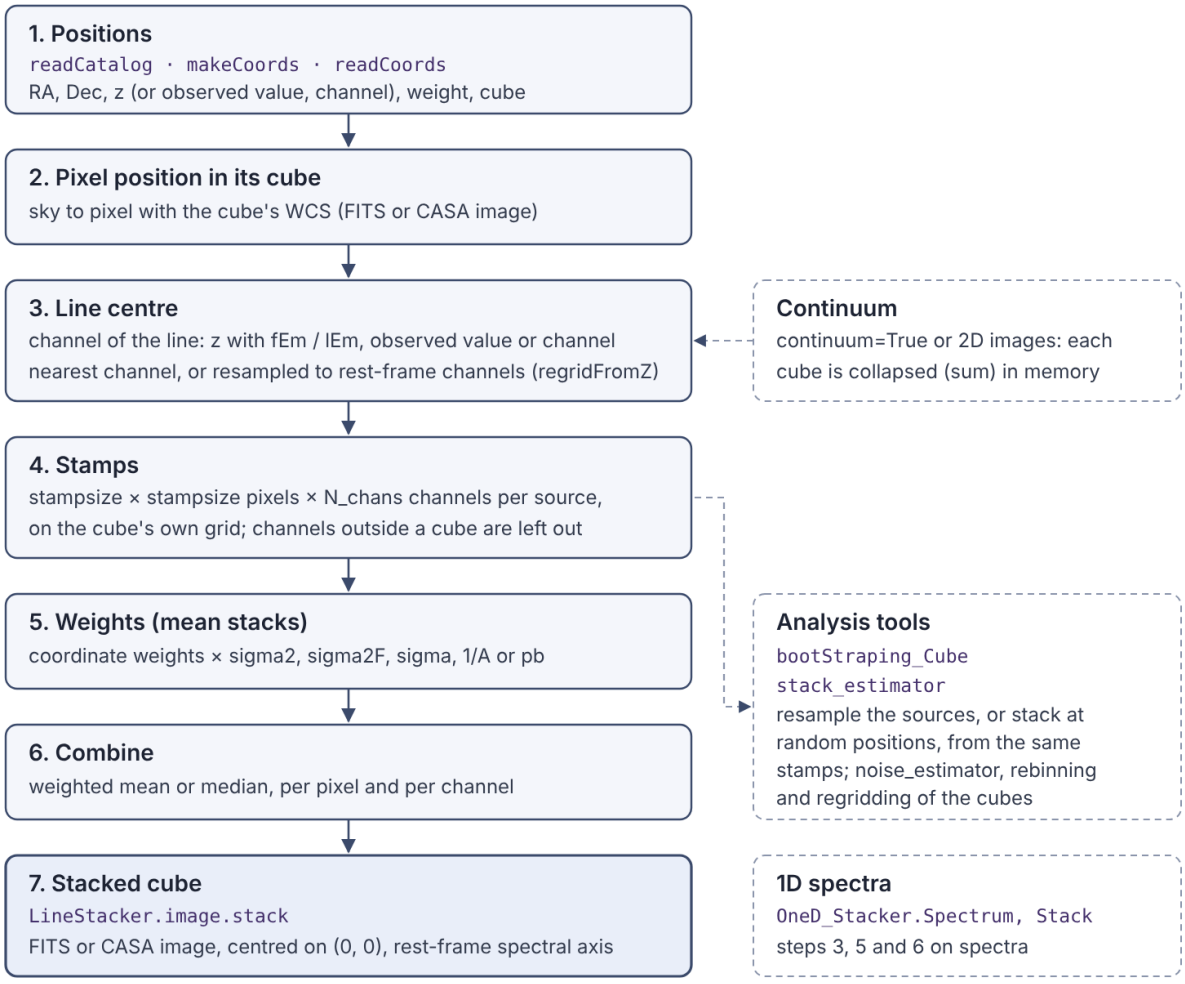
LineStacker reports what it does: positions read, the full path of every file it writes, progress bars. `LineStacker.verbose = False` silences it; every function that prints also takes `verbose=True` or `verbose=False` for one call. Possible problems (cubes collapsed for continuum stacking, sources with no valid noise, cubes on different pixel grids…) are Python warnings and show either way.

CHAPTER

TWO

HOW STACKING WORKS

Stacking averages the data of many sources, each at its own position and redshift, so that a signal too faint to be seen in any single source shows up in the average. Noise that is independent between sources goes down as $1/\sqrt{N}$ for N sources of similar depth.



2.1 Stacking in data cubes

For every source, *`LineStacker.image.stack()`*:

1. **Takes its position**, from a catalogue, Python arrays or coordinate files (*Stacking positions*): right ascension, declination, a spectral value (the redshift, by default) and a weight, and the cube the source lies in.
2. **Converts the position to pixels** with the WCS of that cube. Cubes can be FITS files or CASA images, in any axis order.

3. **Finds the channel of its line.** With `spectralMethod='z'` (the default) the line is at the rest frequency `fEm` (or rest wavelength `lEm`) redshifted to the source's z; `'obsSpec'` gives the observed frequency, wavelength or velocity directly, and `'channel'` the channel. The stack uses the nearest channel, one cube channel per stack channel, as LineStacker always did. With `regridFromZ=True` each spectrum is instead resampled so that all sources share the same rest-frame channel width, which corrects the narrowing of the observed lines with redshift.
4. **Cuts a stamp** of `stampsize` × `stampsize` pixels and `N_chans` channels, centred on the source and on its line. Stamps are cut on the cube's own pixel grid: LineStacker does not reproject cubes. Channels that fall outside a source's cube are left out of the stack for that source.
5. **Weights it** (mean stacks): the weight of the coordinate, times the weighting asked for: the inverse noise variance of the stamp (`'sigma2'`, or per channel `'sigma2F'`), the inverse noise (`'sigma'`), one over the line amplitude (`'1/A'`), or the squared primary-beam response (`'pb'`).
6. **Combines all stamps**, pixel by pixel and channel by channel: a weighted mean, or a median.

The stacked cube is written as a FITS file or a CASA image, with sky axes as offsets from the stacked position and, for `spectralMethod='z'`, a spectral axis in the rest frame: the line at `fEm`. See *Stacking cubes* for the options and for what the output header contains.

For continuum stacking (`continuum=True`), steps 3 and 4 work on images: cubes are collapsed over their channels in memory (a sum by default), and images without a spectral axis are used as they are.

2.2 Stacking spectra

`LineStacker.OneD_Stacker.Stack()` does steps 3, 5 and 6 on spectra that were already extracted: every spectrum is a *`LineStacker.OneD_Stacker.Spectrum`* that knows where its line is (a channel, a velocity, a frequency, a redshift, or a Gaussian fit), and the spectra are aligned on their lines and averaged. See *Stacking spectra*.

2.3 Testing a stack

A stacked line can be dominated by a few bright sources, or be an artefact of the noise. The analysis tools (*Analysis tools*) resample the stacked sources (bootstrapping), find the sources that contribute most (subsampling), and stack random positions around the sources to measure how large a stacked signal noise alone gives.

CHAPTER

THREE

STACKING POSITIONS

Every stacking function takes a list of *LineStacker.Coord*, one per source. There are three ways to make it; all give the same coordinates.

3.1 From a catalogue

LineStacker.readCatalog() reads the whole sample from one table, one row per source:

```
ra,dec,z,weight,image
53.1475,-27.7826,6.6036,1,cubes/source_01.fits
53.1612,-27.7791,6.5899,1,cubes/source_02.fits
03:32:41.31,-27.46.12.4,6.6210,2,cubes/source_03.image
```

```
coords = LineStacker.readCatalog('sources.csv')
```

- **Formats**: CSV (with a header line), ECSV, FITS tables, VOTable and anything else `astropy.table` reads (`format=` if it cannot be guessed), or an astropy `Table`, a pandas `DataFrame` or a dict of columns.
- **Columns**: `ra`, `dec`, `z` (optional), `weight` (optional) and `image`. Names are matched without regard to case, and common alternatives are found: `RAJ2000`/`DEJ2000`, `ra_deg`/`dec_deg`, `redshift`, `zspec`, `cube`, `file`... Other names can be given: `readCatalog(cat, ra='alpha', z='zspec', image='file')`.
- **Positions** are in degrees (or `unit='rad'`), in the unit of the column when the table has units (FITS, ECSV, VOTable), or sexagesimal strings with RA in hours (`03:32:41.31`, `03h32m41.31s`, or CASA's dotted declination `-027.46.12.4`). Each value is read on its own, so one column can mix forms. The frame is ICRS unless `frame='fk5'` (J2000) or another equatorial frame is given.
- **z** is the spectral value of each source: its redshift for the default `spectralMethod='z'`, or the observed frequency, wavelength or velocity for `'obsSpec'`, or the channel for `'channel'`. Without a `z` column the positions are for continuum stacking.
- **image** names the cube of each source. Relative paths are looked up next to the catalogue file, then from the working directory; `imagedir=` sets another folder.

The cube of a source is never guessed from its position: it is the one the catalogue names. If the catalogue has no `image` column, the stacking function must be given a single cube (`imagenames=['field.fits']`), which then holds every source.

3.2 From Python arrays

LineStacker.makeCoords() builds the same coordinates from lists, NumPy arrays, astropy quantities or a `SkyCoord`, for example in a notebook:

```
import numpy as np
from astropy.coordinates import SkyCoord
```

(continues on next page)

(continued from previous page)

```
coords = LineStacker.makeCoords(ra_deg, dec_deg, z=redshifts,
                                image=cube_names)
coords = LineStacker.makeCoords(SkyCoord(ra, dec, unit='deg'), z=redshifts,
                                image='field.fits')
coords = LineStacker.makeCoords(x_pix, y_pix, z=channels, image='cube.fits
 ↪',
                                type='pixel')
```

A single value (`image='field.fits'`, `weight=1`) is used for every source. A `SkyCoord` in another frame, such as Galactic, is converted.

3.3 From coordinate files

`LineStacker.readCoords()` reads the format of LineStacker 1.x: one text file per cube, one line per source, with `ra, dec, z` and optionally a weight (`ra, dec` and an optional weight with `lineON=False` for continuum):

```
03:30:34.79, -027.45.34.566, 7.22611675851
53.1475, -27.7826, 6.6036, 2.0
```

```
coords = LineStacker.readCoords(['cube_1_coord.txt', 'cube_2_coord.txt'])
LineStacker.image.stack(coords, imagenames=['cube_1.image', 'cube_2.image
 ↪'],
                         fEm=fEm)
```

Positions can be in degrees, in sexagesimal with RA in hours (colons, spaces, `h m s` letters or CASA's dots), or pixels with `type='pixel'`. Commas, spaces and tabs are tried as delimiters; empty lines and lines starting with `#` are skipped. The sources of each file belong to the image of the same name (`cube_1_coord.txt` to `cube_1.fits`, or the name given with `images=`) if that image is in `imagenames`, and otherwise to the image at the same position in `imagenames` as the file in the list.

3.4 Which cube a source belongs to

The stacking functions take the list of cubes as `imagenames`. When every coordinate names its cube (catalogue `image` column, `makeCoords(image=...)`), `imagenames` can be left out: the cubes are those the coordinates name. A coordinate whose cube is not in `imagenames` is an error, so a source is never stacked from the wrong cube.

`coords=None` stacks the centre of every cube in `imagenames` (spatially and spectrally).

3.5 Weights

The `weight` of a coordinate multiplies its contribution to mean stacks; the automatic weightings of *`LineStacker.image.stack()`* (`'sigma2'`, `'1/A'`...) multiply it. Weights must be finite and not negative; a weight of 0 leaves the source out of mean stacks. Medians ignore weights.

CHAPTER

FOUR

STACKING CUBES

LineStacker.image.stack() stacks data cubes, FITS files or CASA images (both can be mixed), around the sources of a coordinate list.

```
import LineStacker
import LineStacker.image

coords = LineStacker.readCatalog('sources.csv')
stacked = LineStacker.image.stack(
    coords,
    fEm=1897420620253.1646,    # rest frequency of the line (Hz)
    stampsize=16,              # pixels
    N_chans=31,                # channels of the stack
    outfile='stack.fits')
peak, spectrum, cube, missing = stacked
```

It returns the central pixel at the central channel, the spectrum of the central pixel, the whole stacked cube as an array ordered (x, y, Stokes, channel), and the number of sources missing from each channel (their cube does not cover it).

4.1 Centring the lines

`spectralMethod` says what the spectral value of each coordinate (its `z`) is:

`spectralMethc`	spectral value	also needed
`'z'` (default)	redshift of the line	`fEm` (frequency cubes) or `lEm` (wavelength cubes)
`'obsSpec'`	observed frequency, wavelength or velocity of the line	
`'channel'`	channel of the line (0-based)	

`fEm`, `lEm` and observed values are in the spectral unit of the first cube (Hz for CASA images, Angstrom for MUSE cubes, for example). The cubes must all have the same kind of spectral axis (all frequency, or all wavelength, or all radio velocity); other units are converted (a cube in GHz stacks with cubes in Hz). Cubes whose spectral axis runs the other way are read backwards, so all spectra share the orientation of the first source's cube. Non-linear spectral axes (`WAVE-LOG`, `FREQ-TAB...`) are refused for line stacks, whatever `spectralMethod`, since the stack could not be given a correct spectral axis; regrid such cubes to a linear axis first. Cubes in different spectral reference frames (`SPECSYS`: LSRK, TOPOCENT...), or with `SPECSYS` in some cubes only, give a warning: LineStacker does not convert frames, so regrid such cubes to one frame first (with CASA's `imregrid`, or `tclean(outframe=...)`).

4.1.1 Intensity units

The cubes' pixel units (`BUNIT`) are compared before stacking. Units that differ only by a scale are converted to the unit of the first cube that has one (a cube in mJy/beam stacks with cubes in Jy/beam); recognised but physically different units (K and Jy/beam, Jy/pixel and Jy/beam) are an error. Cubes without `BUNIT`, or with a unit astropy cannot read, give a warning and are stacked without conversion.

`N_chans` channels are stacked around the line centre (default: the number of channels of the first cube). By default each source contributes its nearest channel to each stack channel. With `regridFromZ=True` (needs `spectralMethod='z'`) every spectrum is resampled by linear interpolation so that all sources share the same rest-frame channel width: the width of the source at the highest redshift (`regridMethod='scaleToMax'`, default) or the lowest (`'scaleToMin'`). This corrects the narrowing of lines of the same rest-frame width at higher redshift. A stack channel interpolated from a blank channel is blank for that source; one that falls exactly on a channel takes its value.

4.2 Weights

With `method='mean'` each stamp is weighted, channel by channel, by the weight of its coordinate times the weighting asked for:

`weighting`	weight multiplied by
`None` (default)	1: only the coordinates' weights
`'sigma2'`	$1/\sigma^2$, σ being the standard deviation of the source's stamp
`'sigma2F'`	$1/\sigma^2$ computed channel by channel
`'sigma'`	$1/\sigma$ (as for `'sigma2'`)
`'1/A'`	1/A, A being the line amplitude: the maximum of the central-pixel spectrum. Only for lines visible source by source
`'pb'`	the square of the primary-beam response at the source (see below)

σ is computed from the stamp pixels that are not blank and lie outside `maskradius` pixels of the source; set `maskradius` so that the source does not count as noise. For `'sigma'` the default mask radius is twice the beam's major axis. Sources (or channels) without valid noise pixels, or without a positive line amplitude for `'1/A'`, get weight 0, with a warning. Weights must be finite and not negative.

`method='median'` takes, for each pixel and channel, the median over the sources whose cube covers that channel. It ignores weights.

4.2.1 Primary-beam weighting

For primary-beam corrected images, whose noise grows as 1/PB away from the pointing centre, weighting by PB^2 is inverse-variance weighting. `primarybeam` gives the primary beam of each cube:

```
# primary-beam images (e.g. tclean's .pb), one per cube, read at each
# source's position and line channel
LineStacker.image.stack(coords, imagenames=cubes, fEm=fEm,
                        weighting='pb', primarybeam=pb_images)
LineStacker.image.stack(coords, imagenames=cubes, fEm=fEm, weighting='pb',
                        primarybeam={'a.image': 'a.pb', 'b.image': 'b.pb'})

# a Gaussian beam of 12 m dishes, FWHM 1.13 λ/D, centred on the reference
# pixel of each image
pb = LineStacker.image.GaussianPrimaryBeam(dish=12.)
LineStacker.image.stack(coords, imagenames=cubes, fEm=fEm,
                        weighting='pb', primarybeam=pb)
```

A list gives one primary beam per cube, in the order of `imagenames`; one path is enough for a single cube. Any function `f(dx, dy, nu)` returning the response at offsets `dx`, `dy` (radians, east and north on the sky, from the

position of the image reference pixel) and frequency `nu` (Hz) can be used too.

For line stacks, a primary-beam cube is read at the channel of each source's line, so it needs the same kind of spectral axis as the science cube (a single-plane primary beam can always be used). For continuum stacks, the response is the mean of the primary-beam image's non-blank planes at the source.

4.3 Continuum

`continuum=True` stacks images instead of lines. Images without a spectral axis, or with a single channel, are always stacked as continuum, and so are coordinates without a spectral value (a catalogue without `z`). Cubes with a spectral axis are collapsed over their channels in memory, with a warning: `collapse='sum'` (default), `'mean'` or `'median'`; blank channels are ignored. Only the collapsed images are kept in memory. The result is the same list as for lines, with a single channel; the FITS output is 2D when the inputs had no spectral axis.

4.4 Output

`outfile` names the stacked cube: a name ending in `.fits` (or `.fits.gz`) is written as FITS, any other name (`stack.image`) as a CASA image, which needs casatools. `outfile=None` writes nothing. With verbose on (default) LineStacker prints the full path of the file it wrote.

The header of the stacked cube:

- Pixels without valid data in any source are NaN (blank).
- **Sky axes** are offsets from the stacked position, (0, 0) at the stamp centre, with the pixel size and orientation of the first cube (a rotated grid is written with a PC matrix).
- **Spectral axis**: for `spectralMethod='z'`, the rest frame: the line at `fEm` (or `lEm`), channels of the mean rest-frame width, `RESTFRQ` (or `RESTWAV`) set, and `SPECSYS='SOURCE'` if the cubes give a frame; otherwise an axis centred on the mean observed value of the lines.
- **Beam**: `BMAJ`, `BMIN` are the means of the input beams and `BPA` their mean as an axis (179° and 1° give 0°): an approximation of the stack's beam when the input beams differ.
- `BUNIT` of the first cube that has one (other cubes are converted to it, see above); for a continuum stack of collapsed cubes it is per channel ('sum' adds channels), as the `COLLAPSE` keyword says. A continuum stack of cubes with non-linear spectral axes has no spectral axis.
- `NSTACK` (number of sources) and a `HISTORY` line with the method.

LineStacker aligns sources on their centres but does not reproject: cubes with a different pixel size, orientation, projection (SIN, TAN…), celestial frame (ICRS, FK5…) or distortion terms from the first one give a warning, and their stamps are cut on their own grid. Reproject such cubes to a common grid first (CASA's `imregrid`) to use the stacked image spatially. Even the central-pixel spectrum is only comparable between cubes of similar beams and pixel sampling.

Other outputs: `returnWeights=True` writes `weightFile.csv` (ra, dec, z and the weight of each source, per channel if they differ), `saveSubCubes=True` saves the stamps as a NumPy file (`outSubCubes.npy`, with their pixel positions in `outCoordsFile.txt`), and `plotIt=True` plots the stacked spectrum and the number of sources per channel.

4.5 Reading cubes

`LineStacker.image.readCube()` reads a FITS file or a CASA image as a NumPy array ordered (x, y, Stokes, channel), x along RA and y along Dec, as LineStacker uses it, for plots and checks:

```
cube = LineStacker.image.readCube('stack.fits')
spectrum = cube[8, 8, 0, :]
```

FITS cubes can have their data in any HDU (`hdul='auto'` takes the first one with an image, `hdul=1` for MUSE cubes for instance) and their axes in any order. Cubes are read once and cached; a cube is read again

when its file changes (checking costs about 0.1 ms per CASA image each time a cube is used; `LineStacker.cache_check = False` trusts the cache instead). `LineStacker.image.clear_cache()` frees the memory.

Blank pixels (NaN in FITS, masked pixels of CASA images) stay NaN. Stacks leave them out pixel by pixel: a stacked pixel is the mean (or median) of the sources that have a valid value there, and NaN if none has, as are stack channels that no source's cube covers. Pixels that are exactly 0 are data like any other.

CHAPTER

FIVE

STACKING SPECTRA

LineStacker.OneD_Stacker stacks spectra that were already extracted. Each spectrum is a *LineStacker.OneD_Stacker.Spectrum*, which knows where its line is; *LineStacker.OneD_Stacker.Stack()* aligns the spectra on their lines and averages them channel by channel.

```
import numpy as np
import LineStacker.OneD_Stacker as OneD

spectra = []
for i in range(100):
    data = np.loadtxt('data/spectra_%d' % i)        # columns: velocity,␣
↪flux
    centre = np.loadtxt('data/central_velocity_%d' % i)
    spectra.append(OneD.Spectrum(spectrum=data, centralVelocity=centre,
                                 name=str(i)))

stack, nspectra = OneD.Stack(spectra)
```

`Stack` returns the stacked spectrum and the number of spectra in each of its channels. Channels without data (no spectrum covers them, or only blank fluxes or spectra of weight 0) are NaN.

5.1 Spectra

A spectrum is given either as a two-column array, spectral values then flux (shape `[N, 2]` or `[2, N]`): `Spectrum(spectrum=data)`; or as fluxes alone, with or without their spectral values: `Spectrum(amp=flux, velocities=v)`. `velOrFreq='vel'` (default) or `'freq'` says whether the spectral values are velocities or frequencies. Velocities are in km/s. The spectral values must increase or decrease evenly (within 5% of a channel, so values rounded in a text file are fine): channels are located with one channel width, the mean spacing. Resample spectra on other grids (log-spaced, with gaps) to an even grid first.

The line centre comes from, in this order:

- `z=` and `fEmLine=`, the redshift and the rest frequency of the line. For frequency spectra the line is at `fEmLine/(1+z)`. For velocity spectra the velocities are relative to the rest frequency (as a CASA spectrum made with `restfreq=fEmLine`), and the line is at v = c z/(1+z) in the radio convention (`velConvention='radio'`, default, as CASA uses), c z with `'optical'`, or the relativistic velocity with `'relativistic'`;
- `centralVelocity=` or `centralFrequency=`, the observed line centre;
- `centerIndex=`, its channel;
- `fit=True`: a Gaussian fit of the spectrum, which overrides the above.

`weights` is the weight of the spectrum in mean stacks: a number (default 1), an array of one weight per channel, `'sigma2'` ($1/\sigma^2$ of the spectrum, or of the channels outside the line if `fit=True`), or `'1/A'` (one over the line amplitude: the maximum, or the fitted amplitude). Channels whose flux is not finite (NaN, inf) are left out. A spectrum needs at least two channels, and as many spectral values as fluxes. `name` labels the spectrum, for subsampling for instance.

5.2 Stack

```
stack, nspectra = OneD.Stack(spectra, method='median', center='fit',
                             chansStack=41)
```

- `method`: `'mean'` (weighted) or `'median'` (unweighted).
- `center`: `'lineCenterIndex'` (default) uses the centre each spectrum was given; `'fit'` fits a Gaussian to every spectrum; `'zero_vel'` uses the channel at velocity 0; `'center'` the middle channel; an integer, that channel for all spectra.
- `chansStack`: `'full'` (default) covers every channel of every spectrum, aligned on the lines; an integer stacks that many channels around the line. The line centre of every spectrum must be one of its channels (an error says which spectrum it is not).
- `regridFromZ=True` resamples the spectra so that all have the same rest-frame channel width (see *LineStacker.analysisTools.regridFromZ1D()*); every spectrum then needs its `z`. Channels and line centres are scaled by (1+z)/(1+z_ref); with `'scaleToMin'` the last channels of the regridded spectra can fall beyond their data and are then NaN.

CHAPTER

SIX

ANALYSIS TOOLS

LineStacker.analysisTools tests what a stack shows and prepares cubes or spectra for stacking.

6.1 Bootstrapping

Bootstrapping stacks random draws, with replacement, of the stacked sources. The spread of the results shows how much the stack depends on which sources it contains; a few bright sources dominating it give a skewed or multi-peaked distribution.

```
import LineStacker.analysisTools as A

boot = A.bootstraping_OneD(spectra, nRandom=10000, method='median', save=
 ↪'amp')
boot = A.bootStraping_Cube(coords, fEm=fEm, stampsize=16, N_chans=31,
                           nRandom=1000, save='ampAndWidth')
```

Both return a list of `nRandom + 1` entries, the first one for the stack of all sources. `save` decides what is kept from each draw: `'all'` (the whole stack), `'amp'` (its maximum), `'ampAndWidth'` (amplitude and width of a Gaussian fit; fits that do not converge are stored as NaN), and for cubes `'outflow'` (the six parameters of a two-Gaussian fit). For cubes the line is the spectrum summed over the stamp.

6.2 Subsampling

subsample_OneD() stacks random subsets of the spectra and grades every stack; each spectrum collects the grades of the stacks it was part of. Spectra that make stacks better get higher totals.

```
grades = A.subsample_OneD(spectra, nRandom=10000, maxTest=A.maximizeSNR)
best = sorted(grades, key=grades.get, reverse=True)[:10]
```

The grade function is *maximizeSNR()* by default (fitted amplitude over the standard deviation of the valid channels outside the line; 0 when the fit fails or fewer than 2 such channels are left), *maximizeAmp()* (maximum), *maximizeOutflow()* (amplitude $\times$ width^2 of the fainter component of a two-Gaussian fit), or any function of a stacked spectrum. `minSize` and `maxSize` bound the sizes of the subsets.

6.3 Random positions and noise

stack_estimator() repeats a cube stack at random positions, between `lowerLimit` and `upperLimit` radians from each source (5 and 10 beams by default), keeping each source's redshift. The distribution of these stacks is what noise alone gives:

```
std, mean, stacks = A.stack_estimator(coords, fEm=fEm, nRandom=2000, N_
 ↪chans=31)
```

noise_estimator() measures the noise of images or cubes from random pixels, optionally outside a mask, for the whole cube or channel by channel. Blank (NaN) pixels are never drawn; pixels that are exactly 0 are

data. *LineStacker.randomCoords()* draws random pixel positions anywhere in a set of images, and *LineStacker.randomizeCoords()* moves positions randomly by one to a few beams.

6.4 Line widths and redshifts

Lines of the same rest-frame width are narrower at higher redshift, and lines of different widths blur a stack. LineStacker can bring them to a common width before stacking:

- *regridFromZ1D()* (spectra) and `LineStacker.image.stack(..., regridFromZ=True)` (cubes, in memory) resample every source to the same rest-frame channel width; *regridFromZ()* writes the regridded cubes (stamps around each source) to disk, to stack as returned with `spectralMethod='z'`.
- *rebin_OneD()* and *rebin_CubesSpectra()* rebin every source so that its line spans as many channels as the narrowest line, measured by Gaussian fits (or given as `widths`). This needs lines visible source by source.

Rebinned and regridded spectra keep their line centres, spectral axes and per-channel weights in step with the new channels. Blank (NaN) channels are never used to make a value: `rebin_OneD` and `rebin_CubesSpectra` interpolate linearly, and a new channel interpolated from a blank channel is blank; `regridFromZ1D` and `regridFromZ` interpolate with a cubic spline through each stretch of valid channels separately, so only new channels between a valid and a blank channel are blank. New channels beyond the last channel of a source (when its channels are made narrower) are blank too.

6.5 Speed, parallelism and reproducibility

The Monte-Carlo tools (`bootstraping_OneD`, `bootStraping_Cube`, `subsample_OneD`, `stack_estimator`) take:

- `n_jobs`: the number of CPU cores to use (`-1` for all);
- `backend`: `'process'` or `'thread'`. A suitable one is chosen per task: threads for NumPy-bound work (mean and median stacks), processes for Python-bound work (Gaussian fits);
- `seed`: a seed for reproducible draws. With `seed=None` the draws come from NumPy's global random state, so `np.random.seed()` also makes runs reproducible.

All random draws are made before the work is split between workers, so the results do not depend on `n_jobs` or `backend`. Worker processes are started with `spawn`; scripts that use them on macOS or Windows need their code under `if __name__ == '__main__':`. Two process-backed jobs started at the same time from different threads run one after the other (each uses all its workers).

6.6 Fitting

LineStacker.tools.fit has the Gaussian fits the tools use: *GaussFit()*, *DoubleGaussFit()* and *DoublePeakFit()*. They ignore NaN channels and raise *FitError* (a `RuntimeError`) when a fit cannot be done or does not converge.

CHAPTER

SEVEN

EXAMPLES

The repository comes with example data and scripts in `LineStacker/Example`, and a notebook that runs all of them: *Examples notebook*.

7.1 Notebook

`LineStacker/Example/LineStacker_examples.ipynb` runs every example on the example data in about half a minute and writes its files to `Example/notebook_outputs/`. Start it from the `Example` folder:

```
cd LineStacker/Example
jupyter notebook LineStacker_examples.ipynb
```

It covers 1D stacking, bootstrapping and subsampling of spectra; cube stacking from coordinate files, from a CSV catalogue and from Python arrays; line fits, rebinning and bootstrapping of cubes; and continuum stacking with stacks at random positions.

7.2 Scripts

Spectra (`Example/oneD`, run from that folder):

- `basic-example.py` stacks 100 spectra (`data/spectra_<i>`, velocity and flux columns) on their central velocities (`data/central_velocity_<i>`).
- `extended-example.py` takes 50 spectra, 10 of which have lines ten times brighter: a median stack, a bootstrap that shows the stack depends on which sources are drawn, and a subsampling that finds the bright spectra.

Cubes (`Example/cube`): 30 CASA cubes of simulated line emitters in `data` and 50 in `dataExtended`, each with its coordinate file. Give the scripts the data folder:

```
cd LineStacker/Example/cube
python basic_example.py data              # line stack of the 30 cubes
python extended_example.py dataExtended   # spectra, fits, rebinning,␣
 ↪bootstrap
python continuum_example.py data          # continuum stack and random␣
 ↪positions
```

Without a folder, the scripts open a file dialog to choose the coordinate files.

CHAPTER

EIGHT

LINESTACKER EXAMPLES

All LineStacker examples in one notebook, run on the example data shipped with LineStacker (`Example/oneD` and `Example/cube`):

1. **One-dimensional stacking**: a basic stack of 100 spectra, then a median stack with bootstrapping and subsampling.
2. **Cube stacking**: the basic stack, the same stack from a catalogue and from Python arrays, then the extended example (line fits, spectral rebinning, bootstrapping).
3. **Continuum stacking** with the same function, and random-position stacks.

Run it from the `Example` folder. The example cubes are CASA images, so the cube sections need CASA's Python packages (`pip install casatools`); FITS cubes would need no CASA. Everything the notebook writes goes to `notebook_outputs/`.

8.1 Setup

```
import os
import sys
import glob

import numpy as np
import matplotlib.pyplot as plt

# Use the LineStacker of this repository (two folders up) if it is there,
# rather than another installed version.
REPO = os.path.abspath(os.path.join(os.getcwd(), '..', '..'))
if os.path.isfile(os.path.join(REPO, 'LineStacker', '__init__.py')):
    sys.path.insert(0, REPO)

import LineStacker
import LineStacker.OneD_Stacker
import LineStacker.image
import LineStacker.analysisTools
import LineStacker.tools.fit as fitTools

print('LineStacker', LineStacker.__version__, 'from', os.path.
 ↪dirname(LineStacker.__file__))

EXAMPLE = os.getcwd()                          # this notebook's folder
OUT = os.path.join(EXAMPLE, 'notebook_outputs')
os.makedirs(OUT, exist_ok=True)

N_JOBS = -1                                    # Monte-Carlo tools: use all␣
 ↪CPU cores
FEM = 1897420620253.1646                       # [CII] 158 um rest␣
```

(continues on next page)

(continued from previous page)

```
 ↪frequency (Hz)
plt.rcParams['figure.dpi'] = 80
```

```
LineStacker 2.0.0 from LineStacker
```

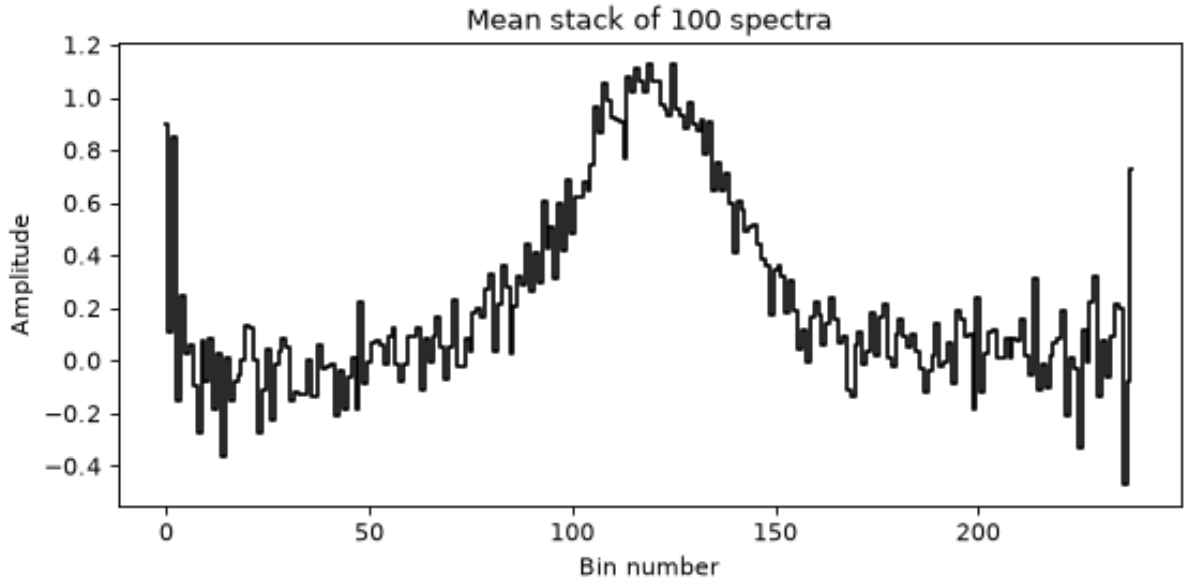
8.2 1. One-dimensional stacking

8.2.1 1.1 Basic stack

100 spectra (`oneD/data/spectra_<i>`), each with its line centre given as a velocity (`central_velocity_<i>`). Every spectrum becomes a `LineStacker.OneD_Stacker.Spectrum`; `Stack` aligns them on their line centre. `Stack` returns the stacked spectrum and the number of spectra contributing to each channel.

```
data = os.path.join(EXAMPLE, 'oneD', 'data')
spectra = [LineStacker.OneD_Stacker.Spectrum(spectrum=np.loadtxt(os.path.
 ↪join(data, 'spectra_%d' % i)),
                                              centralVelocity=np.loadtxt(os.
 ↪path.join(data, 'central_velocity_%d' % i)))
           for i in range(100)]
stacked, n_per_channel = LineStacker.OneD_Stacker.Stack(spectra)

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.step(range(len(stacked)), stacked, where='mid', color='k')
ax.set_xlabel('Bin number')
ax.set_ylabel('Amplitude')
ax.set_title('Mean stack of %d spectra' % len(spectra))
plt.show()
```



8.2.2 1.2 Median stack, bootstrapping and subsampling

50 spectra (`oneD/dataExtended`): 40 have a line of amplitude 1 and 10 (numbers 40 to 49) a line of amplitude 10. Naming the spectra makes the subsampling results readable.

```
data = os.path.join(EXAMPLE, 'oneD', 'dataExtended')
names = ['spectrum_%d' % i for i in range(50)]
spectra = [LineStacker.OneD_Stacker.Spectrum(spectrum=np.loadtxt(os.path.
 ↪join(data, 'spectra_%d' % i)),
                                             centralVelocity=np.loadtxt(os.
 ↪path.join(data, 'central_velocity_%d' % i)),
                                             name=names[i])
           for i in range(50)]
stacked, _ = LineStacker.OneD_Stacker.Stack(spectra, method='median')
stackAmp = np.max(stacked)
print('amplitude of the median stack: %.3f' % stackAmp)
```

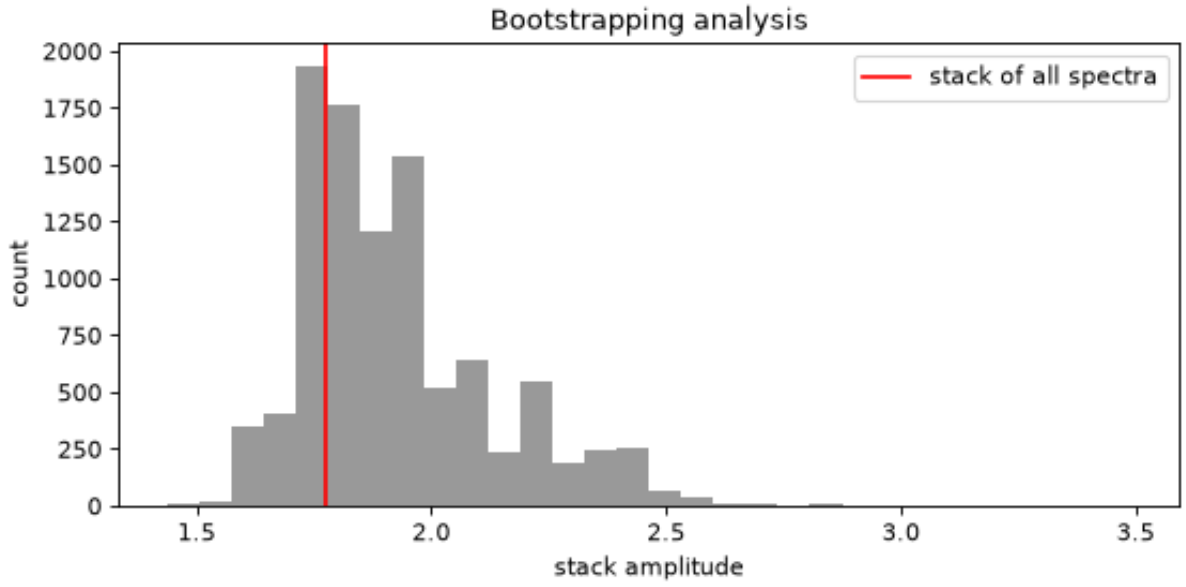
```
amplitude of the median stack: 1.774
```

Bootstrapping: stack spectra drawn with replacement many times, here saving the amplitude of each stack. The original example uses `nRandom=100000`; 10 000 is enough to see the distribution and runs in seconds. `seed` makes the draws reproducible, and the result does not depend on `n_jobs`.

```
bootstrapped = LineStacker.analysisTools.bootstraping_OneD(spectra, save=
 ↪'amp', nRandom=10000,
                                                            method='median',
 ↪ seed=1, n_jobs=N_JOBS)

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.hist(bootstrapped, bins=30, color='0.6')
ax.axvline(stackAmp, color='red', label='stack of all spectra')
ax.set_xlabel('stack amplitude')
ax.set_ylabel('count')
ax.set_title('Bootstrapping analysis')
ax.legend()
plt.show()
```

```
LineStacker: bootstrapping 50 spectra, 10000 iterations
|          | LOADING  100%
```



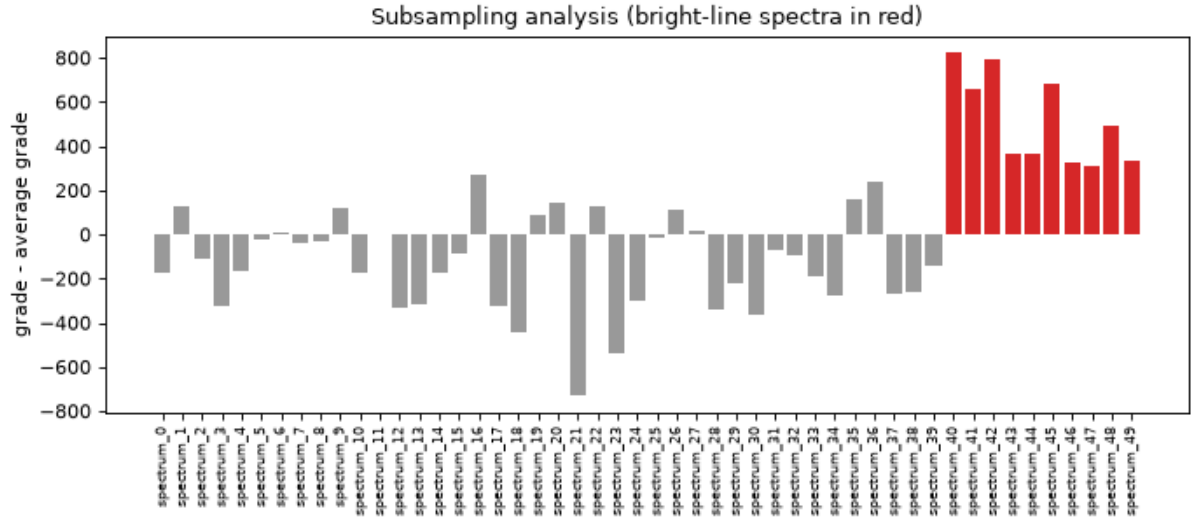
The distribution has several peaks: some draws contain more of the bright spectra than others. **Subsampling** finds which spectra are responsible: random subsets are stacked and graded (by default `maximizeSNR`, the signal to noise of a Gaussian fit), and every spectrum collects the grades of the subsets it was in. The bright spectra come

out on top already with 10 000 iterations.

```
grades = LineStacker.analysisTools.subsample_OneD(spectra, nRandom=10000,␣
↪method='median',
                                                   seed=1, n_jobs=N_JOBS)
meanGrade = np.mean(list(grades.values()))

fig, ax = plt.subplots(figsize=(10, 3.5))
ax.bar(range(len(names)), [grades[n] - meanGrade for n in names],
       color=['0.6'] * 40 + ['tab:red'] * 10)
ax.set_xticks(range(len(names)))
ax.set_xticklabels(names, rotation='vertical', fontsize=7)
ax.set_ylabel('grade - average grade')
ax.set_title('Subsampling analysis (bright-line spectra in red)')
plt.show()
```

```
LineStacker: subsampling 50 spectra, 10000 iterations
|          | LOADING  100%
```



8.3 2. Cube stacking

8.3.1 2.1 Basic stack from coordinate files

The classic input: one coordinate file per cube (`<cube>_coord.txt`, one line `RA, Dec, z` per source), here 30 cubes of the [CII] line. Instead of the file dialog (`LineStacker.readCoordsNamesGUI()`) the files are listed with `glob`. The redshift gives the line centre, so the rest frequency `fEm` is needed. LineStacker prints where it wrote the stacked cube (`LineStacker.verbose`).

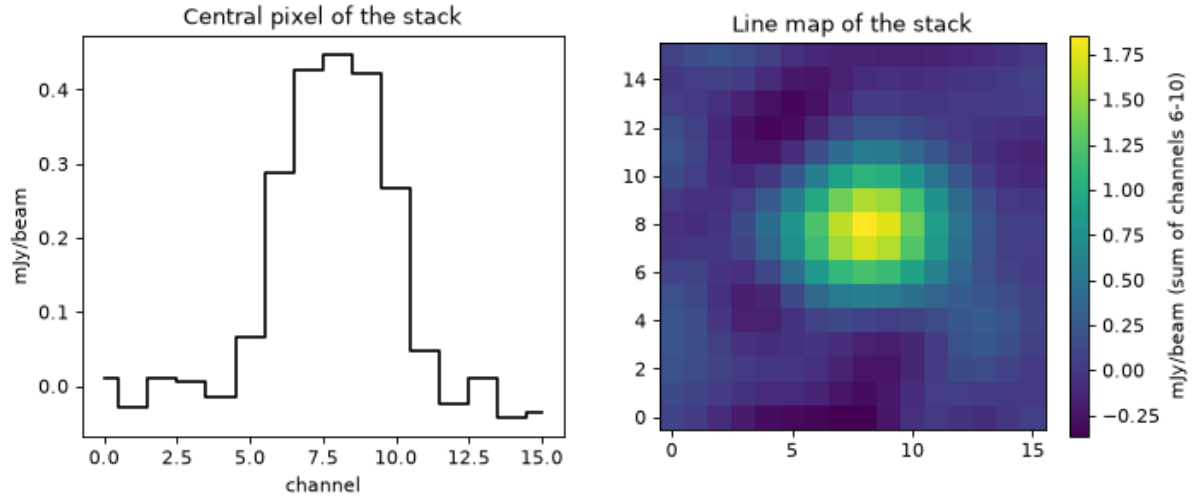
```
coordNames = sorted(glob.glob(os.path.join(EXAMPLE, 'cube', 'data', '*_
↪coord.txt')))
coords = LineStacker.readCoords(coordNames)
imagenames = [c.removesuffix('_coord.txt') + '.image' for c in coordNames]

stacked = LineStacker.image.stack(coords, imagenames=imagenames, fEm=FEM,␣
↪stampsize=16, N_chans=16,
                                  outfile=os.path.join(OUT, 'stackResult.
↪fits'))
centralValue, centralSpectrum, stackCube, missing = stacked
```

```
LineStacker: read 30 positions from 30 coordinate files
LineStacker: mean line stack of 30 sources (16x16 pixels, 16 channels)␣
 ↪written to LineStacker/Example/notebook_outputs/stackResult.fits
```

`stack` returns the central pixel at the central channel, the spectrum of the central pixel, the stacked cube (axes x, y, Stokes, channel) and how many sources miss each channel (sources whose cube does not cover it).

```
line = slice(6, 11)                                    # channels around␣
 ↪the line
fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
axes[0].step(range(16), centralSpectrum * 1e3, where='mid', color='k')
axes[0].set_xlabel('channel')
axes[0].set_ylabel('mJy/beam')
axes[0].set_title('Central pixel of the stack')
im = axes[1].imshow(stackCube[:, :, 0, line].sum(axis=2).T * 1e3, origin=
 ↪'lower')
fig.colorbar(im, ax=axes[1], label='mJy/beam (sum of channels 6-10)')
axes[1].set_title('Line map of the stack')
plt.show()
```



8.3.2 2.2 The same stack from a catalogue

One table for the whole sample, one row per source, with the cube of each source named in an `image` column. `readCatalog` reads CSV, ECSV, FITS tables, VOTable (anything `astropy.table` reads), an astropy `Table` or a pandas `DataFrame`. Positions can be in degrees or sexagesimal (RA in hours, CASA's `-027.40.59.866` included). Relative cube names are looked up next to the catalogue first.

Here the catalogue is written from the coordinate files above, with cube names relative to the catalogue; normally it comes from your own catalogue (TOPCAT, a database query...).

```
catalog = os.path.join(OUT, 'basic_catalog.csv')
with open(catalog, 'w') as fh:
    fh.write('ra,dec,z,image\n')
    for c in coords:
        cube = c.image.removesuffix('.fits') + '.image'
        fh.write('%.8f,%.8f,%.10f,%s\n' % (c.ra.deg, c.dec.deg, c.z, os.
 ↪path.relpath(cube, OUT)))
print(''.join(open(catalog).readlines()[:4]) + '...')
```

```
ra,dec,z,image
52.76729167,-27.68329611,7.2657160664,../cube/data/basicExampleCube_10071.
 ↪image
52.81320833,-27.57057417,7.2219053565,../cube/data/basicExampleCube_14115.
 ↪image
52.64495833,-27.75960167,7.2261167585,../cube/data/basicExampleCube_16612.
 ↪image
...
```

```
coordsCat = LineStacker.readCatalog(catalog)
stackedCat = LineStacker.image.stack(coordsCat, fEm=FEM, stampsize=16, N_
 ↪chans=16, outfile=None)  # no imagenames needed
print('same stack as from the coordinate files:', np.
 ↪allclose(stackedCat[2], stackCube, rtol=1e-6, atol=1e-9))
```

```
LineStacker: read 30 positions in 30 cubes from basic_catalog.csv
LineStacker: mean line stack of 30 sources (16x16 pixels, 16 channels)␣
 ↪done (outfile=None: not written to disk)
same stack as from the coordinate files: True
```

Columns can have other names: `readCatalog(cat, ra='RAJ2000', dec='DEJ2000', z='zspec', image='cube')` (common names such as `RAJ2000`, `redshift`, `cube` are found automatically). An optional `weight` column gives weights for mean stacks. Without a `z` column the positions are for continuum stacking. Without an `image` column, all sources must lie in the single cube given as `imagenames`; LineStacker never guesses which cube a source belongs to.

8.3.3 2.3 The same stack from Python arrays

`makeCoords` builds the coordinates from lists, numpy arrays, astropy quantities or a `SkyCoord`; `image` can be one cube for all sources or one per source.

```
ra = np.array([c.ra.deg for c in coords])        # e.g. columns of your␣
 ↪own catalogue
dec = np.array([c.dec.deg for c in coords])
z = np.array([c.z for c in coords])

coordsArr = LineStacker.makeCoords(ra, dec, z=z, image=imagenames)
stackedArr = LineStacker.image.stack(coordsArr, fEm=FEM, stampsize=16, N_
 ↪chans=16, outfile=None)
print('same stack:', np.allclose(stackedArr[2], stackCube, rtol=1e-6,␣
 ↪atol=1e-9))
```

```
LineStacker: made 30 coordinates
LineStacker: mean line stack of 30 sources (16x16 pixels, 16 channels)␣
 ↪done (outfile=None: not written to disk)
same stack: True
```

8.3.4 2.4 Extended example: line profiles, rebinning and bootstrapping

50 cubes (`cube/dataExtended`) whose lines have different widths. First the stack, and the spectrum of each cube (central 5x5 pixels, read with `LineStacker.image.readCube`, which reads FITS files and CASA images alike).

```
coordNames = sorted(glob.glob(os.path.join(EXAMPLE, 'cube', 'dataExtended',
 ↪ '*_coord.txt')))
coords = LineStacker.readCoords(coordNames)
```

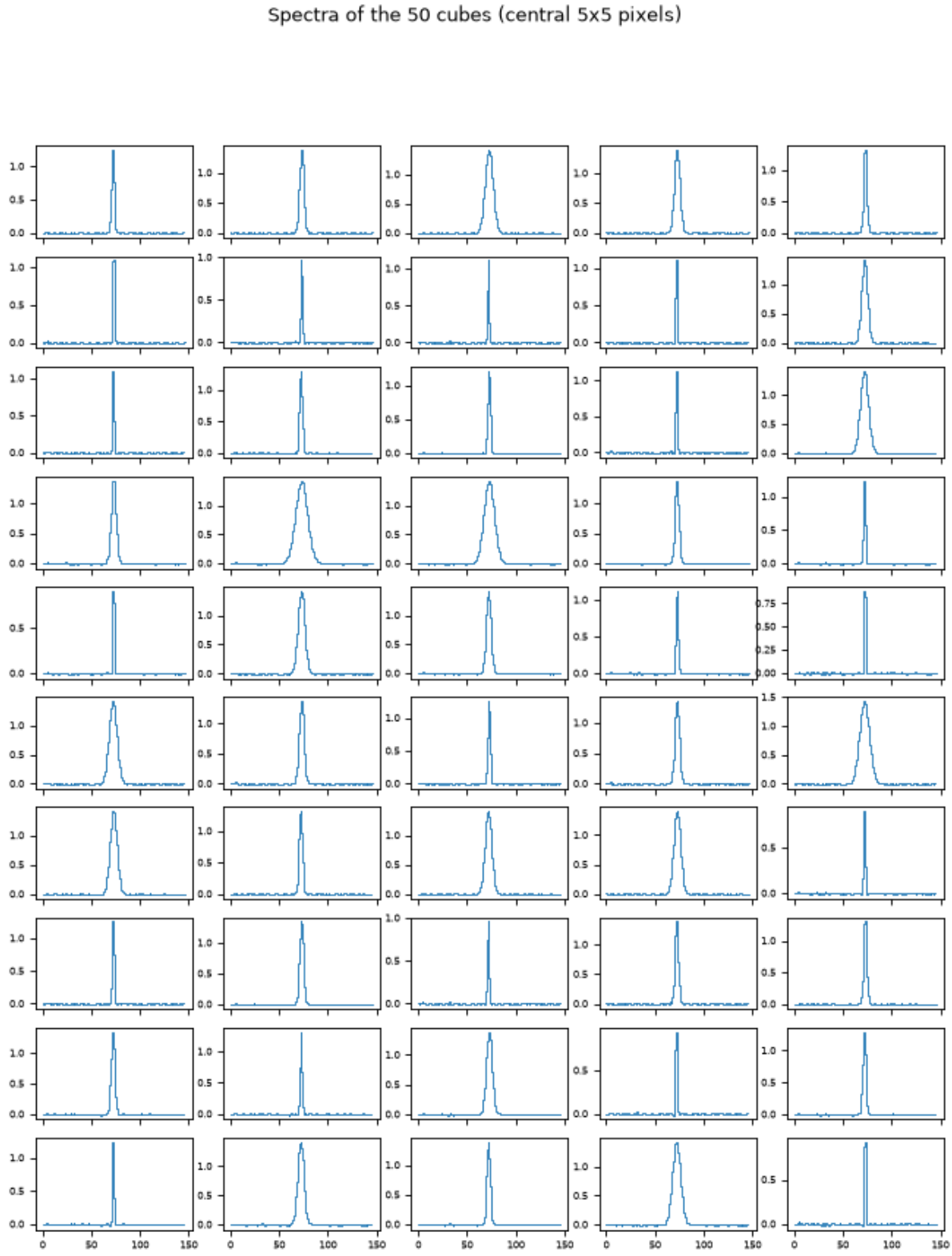
(continues on next page)

(continued from previous page)

```
imagenames = [c.removesuffix('_coord.txt') + '.image' for c in coordNames]
stacked = LineStacker.image.stack(coords, imagenames=imagenames, fEm=FEM,␣
↪N_chans=16, stampsize=16,
                                  outfile=os.path.join(OUT, 'stackResult_
↪extended.fits'))

fig, axes = plt.subplots(10, 5, figsize=(10, 12), sharex=True)
for ax, name in zip(axes.ravel(), imagenames):
    pix = LineStacker.image.readCube(name)
    c = pix.shape[0] // 2
    ax.step(range(pix.shape[3]), pix[c - 2:c + 3, c - 2:c + 3, :, :].
↪sum(axis=(0, 1, 2)), where='mid', lw=0.8)
    ax.tick_params(labelsize=6)
fig.suptitle('Spectra of the 50 cubes (central 5x5 pixels)')
plt.show()
```

```
LineStacker: read 50 positions from 50 coordinate files
LineStacker: mean line stack of 50 sources (16x16 pixels, 16 channels)␣
↪written to LineStacker/Example/notebook_outputs/stackResult_extended.fits
```



The spectrum of the stack, summed over the stamp, fitted with one and with two Gaussians. The narrow core plus broad wings come from stacking lines of different widths.

```
import warnings
from scipy.optimize import OptimizeWarning

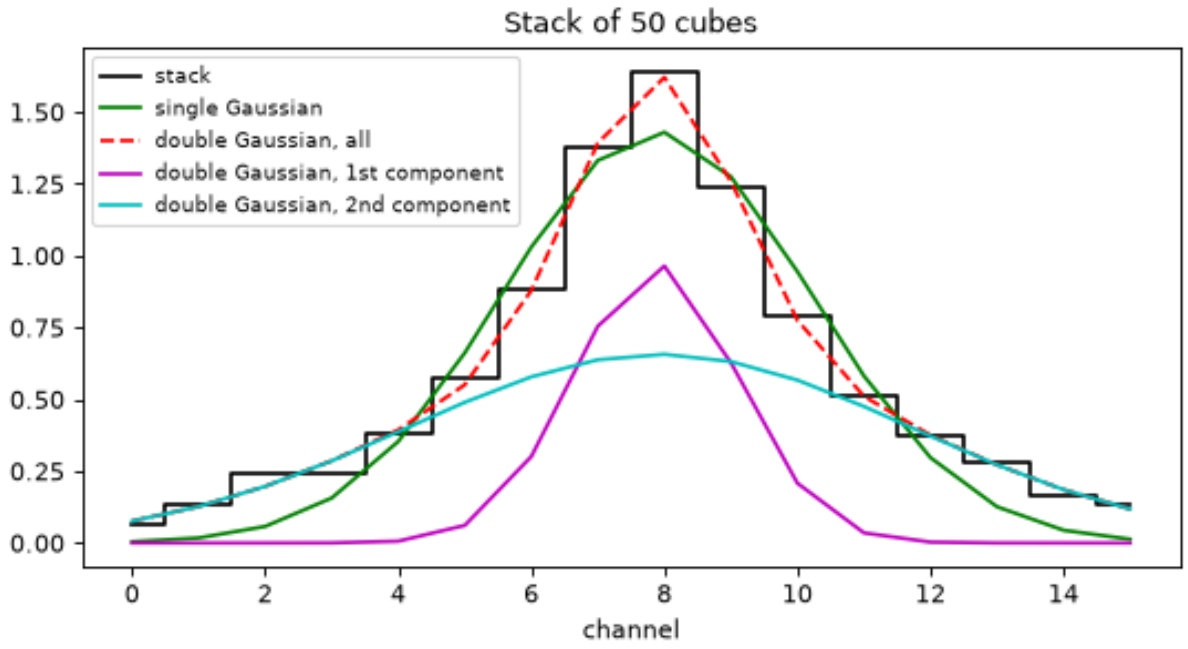
def plot_fits(spectrum, x, title, xlabel):
```

(continues on next page)

(continued from previous page)

```
    with warnings.catch_warnings():
        # a double Gaussian is degenerate for a single-component line
        warnings.simplefilter('ignore', OptimizeWarning)
        single = fitTools.GaussFit(fctToFit=spectrum)
        double = fitTools.DoubleGaussFit(fctToFit=spectrum,␣
↪returnAllComp=True)
    fig, ax = plt.subplots(figsize=(8, 3.8))
    ax.step(x, spectrum, 'k', where='mid', label='stack')
    ax.plot(x, single, 'g', label='single Gaussian')
    ax.plot(x, double[0], 'r--', label='double Gaussian, all')
    ax.plot(x, double[1], 'm', label='double Gaussian, 1st component')
    ax.plot(x, double[2], 'c', label='double Gaussian, 2nd component')
    ax.set_xlabel(xlabel)
    ax.set_title(title)
    ax.legend(fontsize=8)
    plt.show()

integrated = stacked[2].sum(axis=(0, 1, 2))
plot_fits(integrated, range(len(integrated)), 'Stack of 50 cubes', 'channel
↪')
```



Spectral rebinning: each cube is resampled so that all lines get the width of the narrowest one (the widths come from Gaussian fits of each cube's spectrum). The rebinned cubes are written to `notebook_outputs/rebinned`, then stacked; the stacked line is now a single Gaussian. The velocity axis comes from the header of the stacked cube.

```
rebinned = LineStacker.analysisTools.rebin_CubesSpectra(coords, imagenames,
↪ outdir=os.path.join(OUT, 'rebinned'))
outfile = os.path.join(OUT, 'stackResult_rebinned.fits')
rebinnedStack = LineStacker.image.stack(coords, imagenames=rebinned,␣
↪fEm=FEM, N_chans=16, stampsize=16,
                                        outfile=outfile)

from astropy.io import fits
h = fits.getheader(outfile)
```

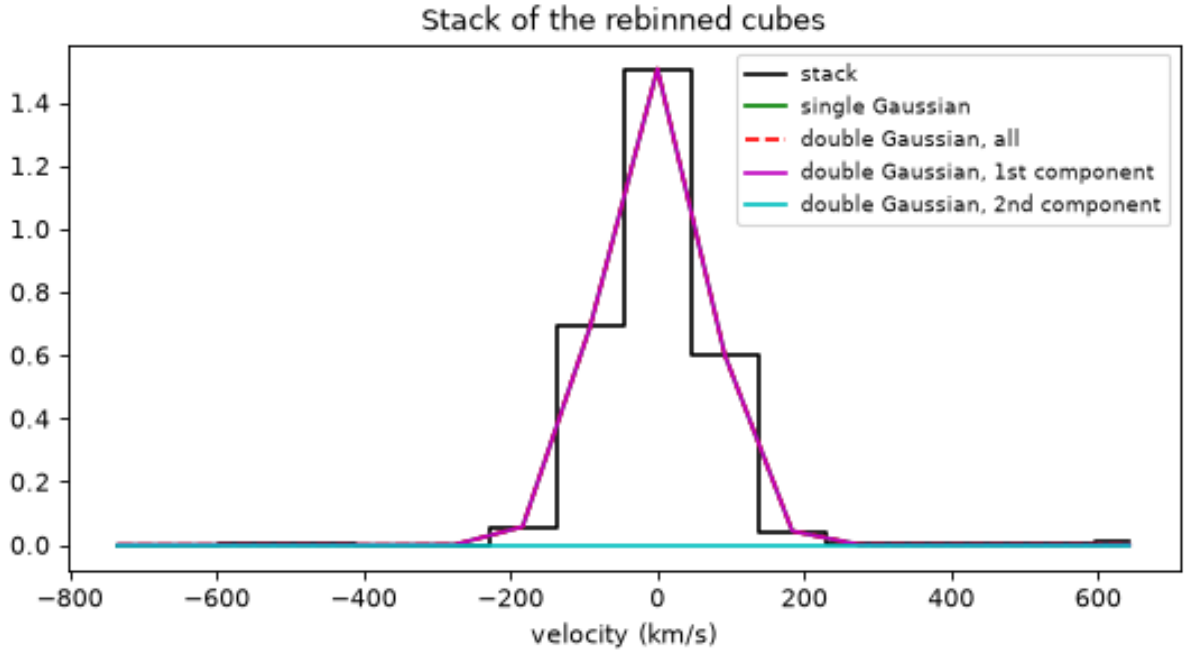
(continues on next page)

(continued from previous page)

```
channels = np.arange(h['NAXIS3'])
freq = h['CRVAL3'] + (channels + 1 - h['CRPIX3']) * h['CDELT3']
velocity = 299792.458 * (h['RESTFRQ'] - freq) / h['RESTFRQ']      # radio␣
 ↪convention, km/s
plot_fits(rebinnedStack[2].sum(axis=(0, 1, 2)), velocity, 'Stack of the␣
 ↪rebinned cubes', 'velocity (km/s)')
```

```
LineStacker: rebinning 50 cubes to the narrowest line width
|          | LOADING  100%

LineStacker: 50 rebinned cubes written: in LineStacker/Example/notebook_
 ↪outputs/rebinned (extendedExampleCube_10619_SpectralRebinned.image ...␣
 ↪extendedExampleCube_97379_SpectralRebinned.image)
LineStacker: mean line stack of 50 sources (16x16 pixels, 16 channels)␣
 ↪written to LineStacker/Example/notebook_outputs/stackResult_rebinned.fits
```



Bootstrapping the cube stack, saving the amplitude and width of a Gaussian fit of each bootstrapped stack (fits are spread over all cores).

```
boot = LineStacker.analysisTools.bootStraping_Cube(coords,␣
 ↪imagenames=imagenames, fEm=FEM, N_chans=16,
                                                    stampsize=16,␣
 ↪nRandom=2000, save='ampAndWidth',
                                                    seed=1, n_jobs=N_JOBS)
amp, width = np.array(boot[1:], dtype=float).T

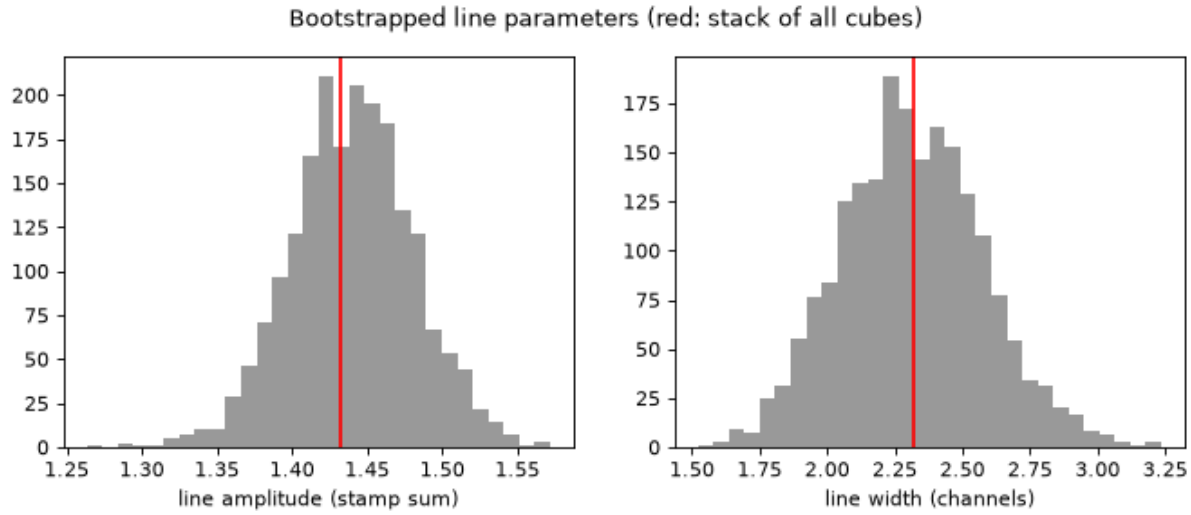
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))
axes[0].hist(amp, bins=30, color='0.6')
axes[0].axvline(boot[0][0], color='red')
axes[0].set_xlabel('line amplitude (stamp sum)')
axes[1].hist(np.abs(width), bins=30, color='0.6')
axes[1].axvline(abs(boot[0][1]), color='red')
axes[1].set_xlabel('line width (channels)')
fig.suptitle('Bootstrapped line parameters (red: stack of all cubes)')
```

(continues on next page)

(continued from previous page)

```
plt.show()
print('amplitude %.4f +- %.4f, width %.2f +- %.2f channels' % (boot[0][0],␣
 ↪np.nanstd(amp), abs(boot[0][1]), np.nanstd(np.abs(width))))
```

```
LineStacker: bootstrapping 50 sources, 2000 iterations
|          | LOADING  100%
amplitude 1.4317 +- 0.0407, width 2.32 +- 0.26 channels
```



8.4 3. Continuum stacking

The same `stack` function, with `continuum=True`. The example cubes have a spectral axis, so LineStacker warns and stacks the images summed over channels (`collapse='sum'`, the default; `'mean'` and `'median'` are possible), made in memory. Images without a spectral axis are stacked as continuum automatically, without warning.

```
coordNames = sorted(glob.glob(os.path.join(EXAMPLE, 'cube', 'data', '*_
 ↪coord.txt')))
coords = LineStacker.readCoords(coordNames)
imagenames = [c.removesuffix('_coord.txt') + '.image' for c in coordNames]

continuum = LineStacker.image.stack(coords, imagenames=imagenames,␣
 ↪stampsize=16, continuum=True,
                                    outfile=os.path.join(OUT,
 ↪'continuumStack.fits'))

fig, ax = plt.subplots(figsize=(4.5, 3.8))
im = ax.imshow(continuum[2][:, :, 0, 0].T, origin='lower')
fig.colorbar(im, ax=ax, label='Jy/beam x channels')
ax.set_title('Continuum stack')
plt.show()
```

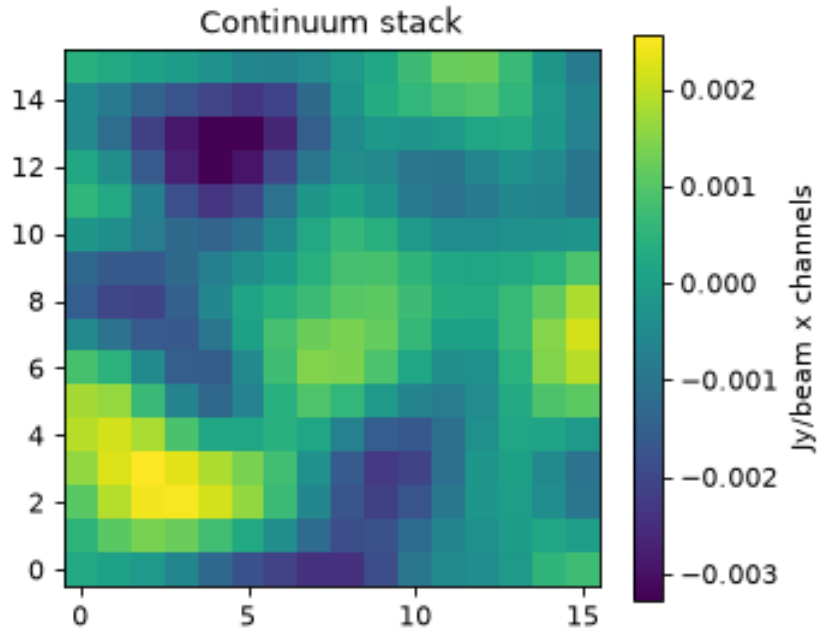
```
LineStacker: read 30 positions from 30 coordinate files
LineStacker: mean continuum stack of 30 sources (16x16 pixels) written to␣
 ↪LineStacker/Example/notebook_outputs/continuumStack.fits
```

```
<cell>:5: UserWarning: continuum stacking (continuum=True): collapsing 30␣
 ↪cube(s) with a spectral axis (sum over channels) in memory
```

(continues on next page)

(continued from previous page)

```
 continuum = LineStacker.image.stack(coords, imagenames=imagenames,␣
↪stampsize=16, continuum=True,
```



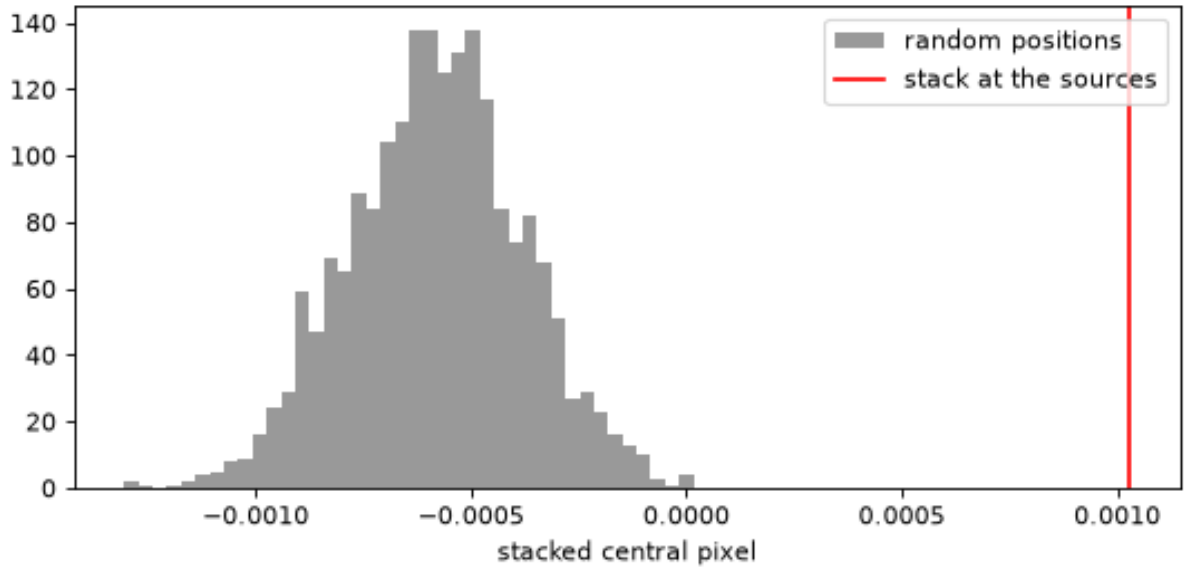
Stacks at random positions tell how significant the stacked value is: each source is moved to a random position (here 4 to 9 microradians, about 0.8” to 1.9”, away; the example cubes are only 6” wide) and the stack is repeated.

```
random = LineStacker.analysisTools.stack_estimator(coords,␣
↪imagenames=imagenames, continuum=True, nRandom=2000,
                                                    lowerLimit=4e-6,␣
↪upperLimit=9e-6, seed=1, n_jobs=N_JOBS)
std, mean, dist = random

fig, ax = plt.subplots(figsize=(8, 3.5))
ax.hist(np.ravel(dist), bins=40, color='0.6', label='random positions')
ax.axvline(continuum[0], color='red', label='stack at the sources')
ax.set_xlabel('stacked central pixel')
ax.legend()
plt.show()
print('stack %.3g, random positions %.3g +- %.3g' % (continuum[0], mean,␣
↪std))
```

```
LineStacker: stacking 2000 sets of random positions around 30 sources
|          | LOADING 25%
|          | LOADING  100%
stack 0.00103, random positions -0.000582 +- 0.000201
```

```
<cell>:1: UserWarning: continuum stacking (continuum=True): collapsing 30␣
↪cube(s) with a spectral axis (sum over channels) in memory
  random = LineStacker.analysisTools.stack_estimator(coords,␣
↪imagenames=imagenames, continuum=True, nRandom=2000,
```



The random positions give negative values on average: they fall in the negative sidelobes of the dirty beam around each source, as the example cubes are small. On real data, use limits of several beams (the default, 5 to 10 beams).

8.5 4. Good to know

- `LineStacker.verbose = False` silences LineStacker's messages and progress bars; any function also takes `verbose=True/False`.
- `outfile` names ending in `.fits` are written as FITS, other names (e.g. `stackResult.image`) as CASA images.
- The Monte-Carlo tools take `n_jobs` (cores, -1 for all) and `seed`; results do not depend on `n_jobs`.

CHAPTER

NINE

API REFERENCE

9.1 LineStacker

Stacking positions (*Coord*, *readCatalog()*, *makeCoords()*, *readCoords()*), their conversion to pixels, random positions, and the package settings (*verbose*, *casa_logfile*). The stacking itself is in *LineStacker.image* (cubes) and *LineStacker.OneD_Stacker* (spectra); *LineStacker.analysisTools* tests stacks.

`LineStacker.`**`verbose = True`**

When True (default), LineStacker reports what it does, the files it writes (with their full path), what it reads, and progress bars. Set `LineStacker.verbose = False` to silence it; every function that prints also takes `verbose=True/False` to decide for one call. Warnings about possible problems are shown either way.

`LineStacker.`**`casa_logfile = 'auto'`**

Where CASA writes its log when LineStacker starts CASA itself (only to read or write CASA images; FITS cubes never start CASA):

- `'auto'` (default): one file, `<temporary dir>/linestacker-casa-<user>.log`, instead of a new `casa-<date>.log` in the working directory at every run; a `logfile` set in `~/.casa/config.py` is used instead.
- a path: CASA's log goes there.
- `None`: no log file.
- `'casa'`: CASA's own default (`casa-<date>.log` in the working directory).

Set it before the first CASA image is read. Inside a CASA session, or once casatools/casatasks were imported, CASA's own settings apply.

`LineStacker.`**`cache_check = True`**

Cubes are read once and kept in memory. When True (default) LineStacker checks, every time it uses a cached cube, that its file has not changed (about 0.1 ms per CASA image, a few microseconds per FITS file) and reads it again if it has. Set `LineStacker.cache_check = False` to trust the cache until `LineStacker.image.clear_cache()`.

`LineStacker.`**`round_half_up`**(*x*)

Round like Python 2's round() did (halves away from zero). Python 3's round() rounds halves to even, which would move some stack centres by one channel compared with LineStacker < 2.0.

`class` `LineStacker.`**`Coord`**(*ra*, *dec*, *spec='continuum'*, *weight=1*, *image=0*, *type='physical'*, *imagenumber=-1*, *frame='icrs'*, *z=None*, *index=None*)

Describes a stacking position on an image.

Class used internally to represent coordinates. May describe a physical coordinate or a pixel coordinate.

Init: Creates a coordinate. A pixel coordinate should always specify to which image it belongs. Physical coordinates are astropy angles (as produced by **LineStacker.readCoords**, **readCatalog** and **makeCoords**) or plain numbers in radians, in the equatorial frame given by **frame** (default ICRS).

Parameters

- **ra** – x coordinate of stacking target (RA, or x pixel for pixel coordinates). Also available as `coord.x`.
- **dec** – y coordinate of stacking target (Dec, or y pixel). Also `coord.y`.
- **spec** – spectral coordinate of stacking target (redshift, observed spectral value or channel depending on spectralMethod). Also `coord.z`.
- **weight** – Weight of the source for mean stacking. Default is 1.
- **image** – Cube associated to the source, either its path or its index in the list of images given to the stacking function. Automatically set with **LineStacker.readCoords**.
- **type** – Either 'physical' or 'pixel'
- **imagenumber** – Index of the cube in the image list (set by readCoords to the index of the coordinate file the source was read from).
- **frame** – Equatorial frame of ra, dec: 'icrs' (default), 'fk5' (J2000), 'fk4'...

copy()

Independent copy (weights arrays are copied too).

LineStacker.**readCoordsNamesGUI**()

Opens a file dialog (Tk) to select coordinate files; returns their paths.

LineStacker.**readCoords**(*coordfiles*, *unit='deg'*, *lineON=True*, *delimiter='default'*, *frame='icrs'*, *images='default'*, *type='physical'*, *verbose=None*)

Reads coordinate files (one file per cube, the format of LineStacker 1.x) and returns a list of LineStacker.Coord.

Parameters

- **coordfiles** – List of path to coordinate files. Files in csv format. x and y should be in J2000. If using to stack line, redshift should be in the third column. A weight may be added in the last column to weight positions for mean stacking. If set to **None** stacking position is defined as the center of each cube. Coordinates can either be in degrees (e.g. "10.625, 41.2"), or in hour angle + deg, as hms dms format (e.g. "'00h42m30s', '+41d12m00s'"), or as hour angle + deg without hms nor dms (e.g. "00 42 30, +41 12 00" or CASA's "00:42:30, +41.12.00"). Empty lines and lines starting with # are ignored.
- **unit** – Unit of input coordinates. Allows two values, **'deg'** or **'rad'**.
- **lineON** – Either **True** or **False**, should be set to **True** if doing line stacking. Default is **True**.
- **delimiter** – Delimiter used to separate coordinates, 'default' tries comma, space and tab (in that order).
- **images** – List of images to which each coordfile is assigned, if set to 'default', image names are same as coordfiles, with '.fits' instead of '_coord.txt'. The stacking functions also accept coordinates whose image is not in their imagenames list: the n-th coordinate file is then matched to the n-th image.
- **type** – Coordinate type, can be either 'physical' (default), or 'pixel'
- **verbose** – report how many positions were read (default: LineStacker.verbose)

LineStacker.**makeCoords**(*ra*, *dec=None*, *z=None*, *weight=None*, *image=None*, *unit='deg'*, *frame='icrs'*, *type='physical'*, *verbose=None*)

Builds stacking coordinates directly from Python values (lists, numpy arrays, astropy quantities or a SkyCoord), e.g. in a notebook.

Parameters

- **ra** – Positions, one value per source. Numbers are in **unit** (or pixels if type='pixel'); astropy Angles/Quantities carry their own unit; strings are sexagesimal with RA in hours ('03:31:04.15', '-27:40:59.87', or CASA's '-027.40.59.866'). A SkyCoord can be given as ra, with dec=None.
- **dec** – Positions, one value per source. Numbers are in **unit** (or pixels if type='pixel'); astropy Angles/Quantities carry their own unit; strings are sexagesimal with RA in hours ('03:31:04.15', '-27:40:59.87', or CASA's '-027.40.59.866'). A SkyCoord can be given as ra, with dec=None.
- **z** – Spectral value of each source: the redshift for spectralMethod='z' (the default of LineStacker.image.stack), the observed frequency/wavelength for 'obsSpec', the channel for 'channel'. None for continuum stacking.
- **weight** – Weight of each source for mean stacks (default 1).
- **image** – Cube of each source (path to a FITS file or CASA image), or one path for all sources. None when all sources lie in the single cube given to the stacking function.
- **unit** – 'deg' (default) or 'rad', for plain numbers.
- **frame** – Equatorial frame of the positions: 'icrs' (default), 'fk5' (J2000), 'fk4'... A SkyCoord in any frame (e.g. Galactic) is converted to it.
- **type** – 'physical' (sky positions, default) or 'pixel'.

Returns

list of LineStacker.Coord – to give to LineStacker.image.stack (imagenames can then be omitted when every source names its cube).

LineStacker.**readCatalog**(*catalog*, *ra='ra'*, *dec='dec'*, *z='z'*, *weight='weight'*, *image='image'*, *unit='deg'*, *frame='icrs'*, *imagedir=None*, *format=None*, *verbose=None*)

Reads the stacking positions of all sources from one catalogue table, one row per source. The cube of each source is given in the table (image column); it is never guessed.

Parameters

- **catalog** – Path to a table file: CSV (with a header line), ECSV, FITS table, VOTable or any format astropy.table reads (see **format**). Also accepted: an astropy Table, a pandas DataFrame or a dict of columns.
- **ra** – Column names of the positions (default 'ra', 'dec'; also found: RAJ2000/DEJ2000, ra_deg/dec_deg...). Numbers are in **unit** unless the column has a unit (FITS, ECSV, VOTable); strings are sexagesimal with RA in hours (CASA's '-027.40.59.866' accepted).
- **dec** – Column names of the positions (default 'ra', 'dec'; also found: RAJ2000/DEJ2000, ra_deg/dec_deg...). Numbers are in **unit** unless the column has a unit (FITS, ECSV, VOTable); strings are sexagesimal with RA in hours (CASA's '-027.40.59.866' accepted).
- **z** – Column of the spectral value (default 'z', also 'redshift'): the redshift, or the observed frequency/wavelength or channel for spectralMethod 'obsSpec'/'channel'. If the catalogue has no such column (or z=None) the positions are for continuum stacking.
- **weight** – Column of the weights (optional, default 1).
- **image** – Column naming the cube of each source (FITS file or CASA image; default 'image', also 'cube', 'file'). Relative paths are looked up next to the catalogue first, then from the working directory. Without this column, give the stacking function a single cube.
- **unit** – 'deg' (default) or 'rad', for unit-less numeric positions.
- **imagedir** – Folder to which relative cube names are relative (overrides the rule above).

- **format** – astropy table format, e.g. 'ascii.csv', 'ascii.commented_header' (default: guessed from the file).
- **verbose** – report what was read (default: LineStacker.verbose)

Returns

list of LineStacker.Coord – for LineStacker.image.stack and the analysis tools; imagenames can be omitted there when the catalogue has an image column.

`LineStacker.`**`getPixelCoords`**(*coords*, *imagenames=None*, *hdul='auto'*)

Creates pixel coordinate list from a physical coordinate list and a list of images.

Parameters

- **coords** – A list of LineStacker.Coord coordinates.
- **imagenames** – A list of images' paths (FITS files or CASA images). Optional when every coordinate names its cube (readCatalog, makeCoords).
- **hdul** – index of the FITS HDU holding the cube; 'auto' (default) uses the first HDU containing an image (HDU 0 for ALMA/CASA FITS, 1 for MUSE).

Returns

list of LineStacker.Coord – new coordinates of type 'pixel'; the original coordinates are not modified. Each new coordinate's `index` is the position of the original in `coords`.

`LineStacker.`**`randomCoords`**(*imagenames*, *ncoords=10*, *hdul='auto'*, *margin=0*, *rng=None*)

Random positions anywhere in a set of images: pixel coordinates whose spec is a random channel of their cube, to be stacked with spectralMethod='channel'.

Parameters

- **imagenames** – A list of images paths (or one path).
- **ncoords** – Number of random coordinates. Default is 10.
- **margin** – Minimal distance (pixels) to the image edges, e.g. half the stampsize.
- **rng** – numpy Generator or seed, for reproducible draws.

`LineStacker.`**`randomizeCoords`**(*coords*, *beam*, *maxBeamRange=5*, *imagenames=None*, *hdul='auto'*, *rng=None*, *maxTries=100*)

New positions at a random distance between beam and maxBeamRange*beam from each of the original coordinates (random direction).

Parameters

- **coords** – list of original coordinates (LineStacker.Coord instances)
- **beam** – beam size is radians, new random coordinates will be at a minimum distance beam from the original coordinates.
- **maxBeamRange** – maximum distance from original coordinates at which new coordinates can be located, in units of beams. Default is 5.
- **imagenames** – images the coordinates refer to; when given, new positions are kept inside their image.
- **rng** – numpy Generator or seed, for reproducible draws.

9.2 LineStacker.image

Stacking in the image domain: spectral lines in data cubes, or continuum.

Inputs are FITS files or CASA images (CASA images need casatools), and can be mixed. Line stacking extracts a sub-cube around each source, centred on its line; continuum stacking (`continuum=True`) stacks images. Cubes without a spectral axis (or with a single channel) are always stacked as continuum; cubes with a spectral axis are collapsed in memory when continuum stacking is asked for (with a warning).

What the stacked cube is:

- Stamps are cut on each cube's own pixel grid, centred on the source; cubes are not reprojected. Cubes whose pixels differ in size or orientation, or whose projection, celestial frame or distortions differ, from the first cube's give a warning: reproject them first (e.g. CASA imregrid) to use the stacked image spatially.
- The output sky axes are offsets from the stacked position, (0, 0) at the stamp centre, with the first cube's pixel size and orientation.
- All cubes must have the same kind of spectral axis (FREQ, WAVE, AWAV, VRAD...); units are converted to those of the first cube, in which fEm, lEm and observed spectral values are given. Non-linear spectral axes are refused for line stacks. Different spectral reference frames (SPECSYS) give a warning: LineStacker does not convert frames.
- The beam in the header (BMAJ, BMIN, BPA) is the mean of the input beams, an approximation of the stack's effective beam when they differ.
- Blank pixels (NaN) stay NaN and are left out pixel by pixel; stacked pixels without any valid data are NaN.
- Pixel units (BUNIT) are compared: scaled units are converted to the first cube's (mJy/beam to Jy/beam), recognised but physically different ones are refused, unrecognised ones give a warning.
- A continuum stack of collapsed cubes keeps BUNIT, which is then per channel ('sum' adds channels); the COLLAPSE keyword says how.

LineStacker.image.**stack**(*coords*, *outfile='stackResult.fits'*, *stampsize=32*, *imagenames=None*, *method='mean'*, *spectralMethod='z'*, *weighting=None*, *returnWeights=False*, *maskradius=0*, *fEm=-1*, *lEm=-1*, *N_chans='default'*, *plotIt=False*, *regridFromZ=False*, *regridMethod='scaleToMax'*, *saveSubCubes=False*, *hdul='auto'*, *continuum=False*, *collapse='sum'*, *primarybeam=None*, *verbose=None*)

Performs line (or continuum) stacking in the image domain.

Parameters

- **coords** – A list of LineStacker.Coord (see LineStacker.readCatalog, LineStacker.makeCoords, LineStacker.readCoords). Set to None to stack the centre of every cube.
- **outfile** – Target name for stacked image. Names ending in .fits are written as FITS, other names (e.g. 'stackResult.image') as CASA images (needs casatools). None to skip writing.
- **stampsize** – size of target image in pixels
- **imagenames** – Cubes or images to extract flux from: FITS files or CASA images (both can be mixed), in the same order as the coordinate files. Can be omitted when every coordinate names its cube (readCatalog with an image column, makeCoords(image=...)). All cubes must have the same kind of spectral axis (e.g. all FREQ); different units (Hz, GHz) are converted to those of the first cube. Stamps are cut on each cube's own pixel grid: cubes with other pixel sizes or orientations are not reprojected (a warning says so).
- **method** – 'mean' (weighted) or 'median' (over the sources covering each channel; weights are ignored)
- **spectralMethod** – What the spectral value of each coordinate (its z, the 3rd column of a coordinate file) is. Possible methods are:

  'z': the redshift of the observed line, if used **fEm** (emission frequency) or **lEm** (emission wavelength) must be informed as an argument of this Stack function.

  'obsSpec': the (observed) central frequency, wavelength or velocity, in the spectral unit of the first cube.

  'channel': the channel number (0-based) of the line.

- **weighting** – used if method set to **'mean'**, possible values are:

  'sigma2': weight times 1/sigma**2, sigma being the standard deviation of the source's stamp (pixels outside **maskradius**, blank pixels excluded)

  'sigma2F': the same, computed channel by channel

  '1/A', **/!\ only use if lines are visible pre-stacking**: weight divided by the line amplitude (maximum of the central-pixel spectrum; sources with a non-positive amplitude get weight 0)

  'sigma': weight times 1/sigma (sigma as for 'sigma2'; if maskradius is 0 the mask radius is twice the beam major axis)

  'pb': weight times the squared primary-beam response at each source (needs **primarybeam**); for primary-beam corrected images this is inverse-variance weighting

  None: the weights of the coordinates (1 by default).

  Each weighting multiplies the coordinates' own weights. Weights must be finite and >= 0.

- **returnWeights** – If true weights of each source are written in outfile "weightFile.csv" in format: coord.ra, coord.dec, coord.z, coord.weight.

  If multiple weights exist for one source (for each spectral channel) all are written.

- **maskradius** – allows blanking of centre pixels in weight calculation

- **fEm** – rest emission frequency of the line, should be used if spectralMethod = 'z' and spectral axis of image cube is in frequency (in the unit of the first cube, usually Hz).

- **lEm** – rest emission wavelength of the line, should be used if spectralMethod = 'z' and spectral axis of image cube is in wavelength (in the unit of the first cube, e.g. Angstrom for MUSE).

- **N_chans** – number of channels of the resulting stack, default is number of channels in the first image.

- **plotIt** – direct plot option

- **regridFromZ** – if set to True each source's spectrum is resampled (linear interpolation) so that all sources share the same rest-frame channel width, correcting the redshift dependence of observed line widths. Requires spectralMethod='z'.

- **regridMethod** – Used if regridFromZ is True. Can be set either to 'scaleToMin' or 'scaleToMax'. The common rest-frame channel is the one of the source with the smallest ('scaleToMin') or highest ('scaleToMax') redshift.

- **saveSubCubes** – If set to True (or to str) sub cubes will be saved as a numpy file. (outSubCubes.npy if set to True, user defined is set to str)

- **hdul** – index of the FITS HDU holding the cube; 'auto' (default) picks the first HDU containing an image.

- **continuum** – If True, stack continuum images instead of lines (spectralMethod, fEm, lEm, N_chans and regridFromZ are then not used). Continuum stacking is also used automatically when no image has a spectral axis (or only one channel), and when the coordinates carry no spectral value (readCoords(..., lineON=False)). Cubes with a spectral axis are then collapsed in memory (with a warning), see **collapse**.

- **collapse** – How cubes are collapsed for continuum stacking: 'sum' (default), 'mean' or 'median' over channels. Blanked channels are ignored.

- **primarybeam** – Used by weighting='pb': the primary-beam image of each cube (FITS or CASA, e.g. tclean's .pb), as one path for a single cube, a list in the order of imagenames or a dict {cube: pb image}; the response is read at each source's position (and line channel). Or a function f(dx, dy, nu) returning the response at offsets dx, dy (radians, east and north on the sky) from the image reference pixel and frequency nu

(Hz, None without a frequency axis), e.g. **GaussianPrimaryBeam(dish=12.)**. Entries of a list or dict can also be functions.

- **verbose** – If True, print the path of every file written (stacked cube, weights, sub-cubes). Default: LineStacker.verbose (True unless changed).

Returns

list – [central pixel at the central channel, spectrum of the central pixel, stacked cube (x, y, stokes, channel), number of sources missing from each channel]. For a continuum stack the spectrum has one channel.

class LineStacker.image.**GaussianPrimaryBeam**(*dish=12.0*, *factor=1.13*, *nu0=None*)

Gaussian primary beam (power pattern) of a dish, for weighting='pb': response exp(-4 ln2 r**2 / FWHM**2) with FWHM = factor * c / (nu * dish), r being the offset from the image reference pixel (the pointing centre of single-pointing images). factor=1.13 is the ALMA 12 m value, as in LineStacker.uv.

Parameters

- **dish** – dish diameter in metres (default 12)
- **factor** – FWHM in units of lambda/D (default 1.13)
- **nu0** – frequency (Hz) to use for images without a frequency axis

fwhm(*nu*)

FWHM in radians at frequency nu (Hz).

LineStacker.image.**readCube**(*imagename*, *hdul='auto'*, *collapse=None*)

Reads a cube or image (FITS file or CASA image) as a numpy array ordered (x, y, stokes, channel), x along RA and y along Dec, as LineStacker uses it. Blank pixels are NaN.

Parameters

- **imagename** – path of the FITS file or CASA image
- **hdul** – FITS HDU holding the data ('auto': first HDU with an image)
- **collapse** – None (default) for the full cube, or 'sum', 'mean', 'median' to collapse the spectral axis (blank channels ignored).

LineStacker.image.**clear_cache**()

Forget the cubes read so far: LineStacker reads each cube once and keeps it in memory for the following stacks and analysis tools (a cube is read again when its file changes). This frees that memory.

LineStacker.image.**getFlux**(*imagename*, *hdul='auto'*)

Pixel value at the reference pixel of an image, e.g. the centre of a stacked image (first Stokes plane and channel).

9.3 LineStacker.OneD_Stacker

Stacking of 1D spectra: each spectrum is a *Spectrum*, which knows where its line is; *Stack()* aligns the spectra on their lines and averages them channel by channel. Velocities are in km/s.

class LineStacker.OneD_Stacker.**Spectrum**(*spectrum=None*, *amp=None*, *velocities=None*, *frequencies=None*, *z=None*, *fEmLine=False*, *centerIndex=None*, *centralVelocity=None*, *centralFrequency=None*, *weights=1*, *name=''*, *fit=False*, *velOrFreq='vel'*, *velConvention='radio'*)

One spectrum to stack: its fluxes, optionally its spectral values, and where its line is.

Parameters

- **spectrum** – Spectrum should be of the shape [N,2] or [2,N], where N is the number of spectral bins. The second dimension being the flux and the first the spectral values, which must be evenly spaced (increasing or decreasing; within 5% of a channel).

- **amp** – Instead of defining spectrum one can define only the amplitude (flux).
- **velocities** – In the case where amp is defined it is still possible to define the velocities
- **frequencies** – In the case where amp is defined it is still possible to define the frequencies
- **z** – Redshift, one of the possible way to define central frequency. If using redshift **fEmLine** (emission frequency of the line) should be defined. In addition the spectrum should consist of BOTH amplitudes and spectral information (frequencies or velocities), and not amplitude alone. Velocities are then in km/s relative to the rest frequency fEmLine (as e.g. a CASA spectrum with restfreq=fEmLine), in the convention **velConvention**: the line is at v = c z/(1+z) (radio), c z (optical) or c((1+z)**2-1)/((1+z)**2+1) (relativistic).
- **fEmLine** – Emission frequency of the line, needed if redshift argument is used.
- **centerIndex** – channel index of the line center, another possible way to define line center.
- **centralFrequency** – (observed) Frequency of the line center, another possible way to define line center.
- **centralVelocity** – Observed velocity of the line center, another possible way to define line center.
- **weights** – The weighting scheme to use.

  Can be set to **'1/A'** or **'sigma2'**. If **sigma2** is used std of the entire spectra is used UNLESS, **fit** is set to **True**, in which case the central part around the line is excluded from std calculation (central here means one FWHM on each side of the center of the line).

  Alternatively user input can be used, float or list (or array)
- **name** – name of the spectrum, can allow easier identification
- **fit** – If fit is set to **True**, the spectrum will be fitted with a gaussian to try and identify the line center (as well as amplitude if weight is set to 1/A)
- **velOrFreq** – Defining if spectral dimension is velocity or frequency.
- **velConvention** – Velocity convention of the velocities (km/s), used with z: **'radio'** (default, as CASA and LineStacker.uv), **'optical'** or **'relativistic'**.

velToFreq(*vel*, *z*, *fEmLine*)

Frequency of the velocity vel (km/s) relative to the frequency fEmLine/(1+z) (z=0: the rest frequency), in the spectrum's velConvention. Same unit as fEmLine.

freqToVel(*freq*, *z*, *fEmLine*)

Velocity (km/s) of the frequency freq relative to fEmLine/(1+z) (z=0: the rest frequency), in the spectrum's velConvention.

LineStacker.OneD_Stacker.**velocity_to_frequency**(*vel*, *fRef*, *convention='radio'*)

Frequency (unit of fRef) of velocities vel (km/s) relative to fRef: radio f = fRef (1 - v/c), optical f = fRef / (1 + v/c), relativistic f = fRef sqrt((1 - v/c) / (1 + v/c)).

LineStacker.OneD_Stacker.**frequency_to_velocity**(*freq*, *fRef*, *convention='radio'*)

Velocity (km/s) of frequencies freq relative to fRef (inverse of velocity_to_frequency).

LineStacker.OneD_Stacker.**Stack**(*spectra*, *chansStack='full'*, *method='mean'*, *center='lineCenterIndex'*, *regridFromZ=False*, *regridMethod='scaleToMin'*, *verbose=None*)

Stacks spectra (LineStacker.OneD_Stacker.Spectrum objects), aligned on their lines: weighted mean or median, channel by channel.

Parameters

- **spectra** – List of spectra (LineStacker.OneD_Stacker.Spectrum objects).
- **chansStack** – Number of channels to stack. Set to 'full' to stack all channels from all spectra. User input (int) otherwise
- **method** – stacking method, **'mean'** and **'median'** supported
- **center** – Method to find central frequency of the stack, possible values are:

  "center", to stack all spectra center to center,

  'fit' to use gaussian fitting on the spectrum to determine line center,

  'zero_vel' to stack on velocity=0 bin,

  'lineCenterIndex' use the line center initiated with the spectrum,

  Or directly defined by the user (int)
- **regridFromZ** – if set to True spectra will be regridded to take into account the redshift difference of the different sources. See LineStacker.analysisTools.regridFromZ1D for a more complete description.
- **regridMethod** – Used if regridFromZ is True. Can be set either to **'scaleToMin'** or **'scaleToMax'**. In the first case all spectra are regridded to match the smallest redshift (over-gridding), all spectra are regridded to match the highest redshift in the other case (under-gridding).
- **verbose** – report the regridding (default: LineStacker.verbose)

Returns

tuple – (stacked spectrum, number of spectra in each channel). Channels without valid data (no spectrum covers them, or only blank fluxes or zero weights) are NaN.

9.4 LineStacker.analysisTools

Tools to test stacks and prepare data for stacking: bootstrapping, subsampling, stacks at random positions, noise estimates, rebinning and regridding of lines of different widths and redshifts.

The Monte-Carlo tools (bootstraping_OneD, bootStraping_Cube, subsample_OneD, stack_estimator) accept `n_jobs` (number of CPU cores, -1 for all), `backend` ('process' or 'thread', see LineStacker.tools.parallel), `seed` and `verbose`. All random draws happen before the work is split, so results do not depend on n_jobs. With seed=None the draws come from numpy's global random state, so np.random.seed() still makes runs reproducible.

LineStacker.analysisTools.**bootstraping_OneD**(*spectra*, *nRandom=1000*, *chansStack='full'*, *method='mean'*, *center='lineCenterIndex'*, *save='all'*, *seed=None*, *n_jobs=1*, *backend=None*, *verbose=None*)

Bootstrapping of a 1D stack: stacks of random draws, with replacement, of the spectra. See LineStacker.OneD_Stacker.Stack for the stacking parameters.

Parameters

- **spectra** – a list of LineStacker.OneD_Stacker.Spectrum objects
- **nRandom** – number of bootstrap iterations
- **chansStack** – number of channels to stack, either a fixed number or 'full' for entire spectra
- **method** – stacking method, 'mean' or 'median'
- **center** – Method to center spectra. See LineStacker.OneD_Stacker.Stack for further information on centering methods.

- **save** – data to save at each bootstrap iterations

  possible values are **'all'**, **'amp'** and **'ampAndWidth'**

  'all' saves the full stack at each bootstrap iteration (can use a lot of memory)

  'amp' saves the amplitude of the line (maximum value of the stack) at each bootstrap iteration, fastest

  'ampAndWidth' fits the line with a gaussian and saves the corresponding amplitude and width at each bootstrap iteration (fits that do not converge are stored as [nan, nan]), can be cpu expensive.

- **seed** – seed for reproducible resampling (None uses numpy's global random state)
- **n_jobs** – number of CPU cores to use (-1 for all). Worth it mostly for 'ampAndWidth' and for medians.
- **backend** – 'process' or 'thread', see LineStacker.tools.parallel
- **verbose** – print what is done and a progress bar (default: LineStacker.verbose)

Returns

list – nRandom+1 entries (as chosen by save), the first one for the stack of all spectra.

LineStacker.analysisTools.**bootStraping_Cube**(*coords*, *stampsize=32*, *imagenames=None*, *method='mean'*, *weighting=None*, *maxmaskradius=0*, *fEm=0*, *spectralMethod='z'*, *N_chans=30*, *nRandom=1000*, *save='amp'*, *lEm=-1*, *hdul='auto'*, *maskradius=None*, *seed=None*, *n_jobs=1*, *backend=None*, *continuum=False*, *collapse='sum'*, *verbose=None*)

Bootstrapping of a cube stack: stacks of random draws, with replacement, of the sources. See LineStacker.image.stack for the stacking parameters.

Parameters

- **coords** – A list of LineStacker.Coord of all target coordinates
- **stampsize** – size of target image in pixels
- **imagenames** – Name of images to extract cubes from
- **method** – stacking method, 'mean' or 'median'
- **weighting** – weighting method to use if stacking method is mean

  possible values are 'sigma2', 'sigma2F', '1/A' or None (weights of the coords)

  see LineStacker.image.stack for a complete description of weighting methods

- **maxmaskradius** – radius of the mask used to blank the centre pixels in weight calculation (also accepted as maskradius)
- **fEm** – rest emission frequency of the line (or lEm for wavelength cubes)
- **N_chans** – number of channels of the resulting stack
- **nRandom** – number of bootstrap iterations
- **save** – data to save at each bootstrap iterations

  possible values are **'all'**, **'amp'**, **'ampAndWidth'** and **'outflow'**

  'all' saves the full stack at each bootstrap iteration (can use a lot of memory)

  'amp' saves the amplitude (maximum value of the stack) of the line, at each bootstrap iteration, fastest

'ampAndWidth' fits the line with a gaussian and saves the corresponding amplitude and width at each bootstrap iteration (fits that do not converge are stored as [nan, nan]), can be cpu expensive

'outflow' fits the line with two gaussian components and saves the stack parameters at each bootstrap iteration, can be cpu expensive

for 'amp', 'ampAndWidth' and 'outflow' the line is obtained by summing all pixels inside the stack stamp.

- **seed** – see the module description
- **n_jobs** – see the module description
- **backend** – see the module description
- **continuum** – continuum stacking, see LineStacker.image.stack ('amp' is then the stamp-summed flux; 'ampAndWidth' and 'outflow' need a spectral axis)
- **collapse** – continuum stacking, see LineStacker.image.stack ('amp' is then the stamp-summed flux; 'ampAndWidth' and 'outflow' need a spectral axis)
- **verbose** – print what is done and a progress bar (default: LineStacker.verbose)

Returns

list – nRandom+1 entries (as chosen by save), the first one for the stack of all sources.

LineStacker.analysisTools.**congrid**(*a*, *newdims*, *method='linear'*, *centre=False*, *minusone=False*)

Arbitrary resampling of an array to new dimension sizes (same number of dimensions), with the sampling points of IDL's congrid.

Parameters

- **a** – array to resample
- **newdims** – new size of every dimension
- **method** – **'neighbour'** (closest value), **'nearest'** or **'linear'** (n 1-D interpolations with scipy.interpolate.interp1d), or **'spline'** (scipy.ndimage.map_coordinates, cubic)
- **centre** – True: interpolation points at the centres of the bins; False (default): at their front edges
- **minusone** – False (default): an axis of i samples resized to x is sampled by factors of i/x; True: by (i-1)/(x-1), which avoids extrapolating one sample beyond the input
- **LineStacker's** (*Blank (NaN) values are not handled specially here;*)
- **rebin_OneD).** (*rebinning functions handle them (see*)
- **https** (*From*)
- **2001** (*(Copyright (c)*)
- **Enthought** (*2002*)
- **2003-2017** (*Inc.; Copyright (c)*)
- **reserved.)** (*SciPy Developers. All rights*)

LineStacker.analysisTools.**rebin_OneD**(*spectra*, *verbose=None*)

Rebin a list of spectra so that all lines span as many channels as the narrowest one (widths in channels, from Gaussian fits). **The lines must be visible source by source.** Fluxes are linearly interpolated; a rebinned channel interpolated from a blank (NaN) channel is NaN. The spectral axes, per-channel weights and line centres (centerIndex) follow the new channels.

Parameters

- **spectra** – A list of LineStacker.OneD_Stacker.Spectrum objects

- **verbose** – print what is done (default: LineStacker.verbose)

Returns
list of LineStacker.OneD_Stacker.Spectrum – the rebinned spectra (copies).

LineStacker.analysisTools.**rebin_CubesSpectra**(*coords*, *imagenames=None*, *regionSize=False*, *widths=False*, *outputName='_SpectralRebinned'*, *hdul='auto'*, *outdir=None*, *verbose=None*)

Rebin a list of image-cubes so that all lines have the width of the narrowest one (widths from Gaussian fits, or given). **The lines must be visible source by source.** A rebinned channel interpolated from a blank (NaN) channel is NaN. One coordinate per cube is enough (the first one of each cube is used).

Parameters

- **coords** – A list of LineStacker.Coord (one per image).
- **imagenames** – Name of images to rebin (FITS or CASA images)
- **regionSize** – size (in pixels) of the square region whose summed spectrum is fitted (placed around the source as the stamps of LineStacker.image.stack)

  If set to False spectra will be extracted solely from the coord pixel
- **widths** – widths of the lines

  If set to 'False' the spectra will be fitted with a gaussian to extract the width
- **outputName** – Suffix of the rebinned cube, to add to the original cube name.
- **outdir** – Folder for the rebinned cubes (created if needed); default: next to the originals.
- **verbose** – print where the rebinned cubes are written, and a progress bar (default: LineStacker.verbose)

Returns
list of str – the names of the rebinned cubes (same format as the inputs).

LineStacker.analysisTools.**maximizeAmp**(*spectra*)

Grade of a stacked spectrum: its maximum (blank channels ignored; 0 if every channel is blank).

LineStacker.analysisTools.**maximizeSNR**(*spectra*)

Grade of a stacked spectrum: amplitude of a Gaussian fit over the standard deviation of the valid (finite) channels outside the line (2.35 sigma on each side). 0 if the fit fails, finds no line inside the spectrum, or leaves fewer than 2 valid channels outside it.

LineStacker.analysisTools.**maximizeOutflow**(*spectra*)

Grade of a stacked spectrum for a second (e.g. outflow) component: amplitude x width**2 of the fainter component of a two-Gaussian fit (blank channels ignored; 0 if the fit fails).

LineStacker.analysisTools.**subsample_OneD**(*spectra*, *nRandom=10000*, *maxTest=<function maximizeSNR>*, *seed=None*, *n_jobs=1*, *backend=None*, *minSize=3*, *maxSize=None*, *verbose=None*, ***kwargs*)

Stacks random subsets of the spectra and grades every stack with maxTest; each spectrum collects the grades of the stacks it was part of.

Parameters

- **spectra** – A list of LineStacker.OneD_Stacker.Spectrum objects
- **nRandom** – Number of iterations of the Monte-Carlo process
- **maxTest** – function test to grade the sources build your own, or use existing: **maximizeAmp**, **maximizeSNR**, **maximizeOutflow**. Default is to maximize the signal to

noise ratio. With backend='process' a custom function must be defined at module level (not a lambda) so it can be sent to the workers.

- **minSize** – range of the random subsample sizes (default 3 to all spectra)
- **maxSize** – range of the random subsample sizes (default 3 to all spectra)
- **seed** – see the module description. Gaussian-fit based grades (maximizeSNR, maximizeOutflow) benefit from n_jobs>1.
- **n_jobs** – see the module description. Gaussian-fit based grades (maximizeSNR, maximizeOutflow) benefit from n_jobs>1.
- **backend** – see the module description. Gaussian-fit based grades (maximizeSNR, maximizeOutflow) benefit from n_jobs>1.
- **verbose** – print what is done and a progress bar (default: LineStacker.verbose)
- ****kwargs** – passed to LineStacker.OneD_Stacker.Stack (method, center, chansStack...)

Returns

dict – {spectrum name: summed grade} (spectra without a name are named by their index).

LineStacker.analysisTools.**noise_estimator**(*imagenames*, *nRandom=10000*, *maskradius=0*, *maskCenter=None*, *continuum=True*, *chanToNoise='random'*, *hdul='auto'*, *plotIt=True*, *seed=None*, *verbose=None*)

Estimates the noise of images or cubes from randomly drawn pixels (blank, i.e. NaN or infinite, pixels are never drawn; pixels that are exactly zero are data).

Parameters

- **imagenames** – Path to image (FITS or CASA image), or list of paths
- **nRandom** – Number of iterations
- **maskradius** – Radius of the mask (in pixels)
- **maskCenter** – Center location of the mask (in pixels, [x, y]; default [0, 0]). NB: CENTER OF IMAGE IS DEFINED AS [0,0]
- **continuum** – If set to True points will be taken at random on the any of the spectral channels. Otherwise all channels are probed independently.
- **chanToNoise** – If set to a specific (int) value then noise will be extracted from the specified channel. NB: only works if continuum is set to True.
- **plotIt** – show the (masked) first image
- **seed** – seed for reproducible draws (None uses numpy's global random state)
- **verbose** – print what is done and a progress bar (default: LineStacker.verbose)

Returns

float or numpy array – the standard deviation of the drawn pixels (continuum=True), or one per channel.

LineStacker.analysisTools.**stack_estimator**(*coords*, *nRandom=100*, *imagenames=None*, *stampsize=1*, *method='mean'*, *N_chans=30*, *lowerLimit='default'*, *upperLimit='default'*, *save='central_pixel'*, *fEm=0*, *spectralMethod='z'*, *lEm=-1*, *hdul='auto'*, *seed=None*, *n_jobs=1*, *backend='thread'*, *verbose=None*, ***kwargs*)

Performs stacks at random positions a set number of times. Allows to probe the relevance of stack through stacking random positions as a Monte Carlo process. The random positions keep each source's redshift; positions whose stamp would fall outside the image are drawn again.

Parameters

- **coords** – A list of LineStacker.Coord of all target coordinates.
- **nRandom** – Number of iterations
- **imagenames** – Name of imagenames to stack
- **stampsize** – Size of the stamp to stack (because only the central pixel is extracted anyway, this should be kept to default to enhance efficiency)
- **method** – Method for stacking, see LineStacker.image.stack
- **N_chans** – Number of channels in the stack
- **lowerLimit** – Lower spatial limit (distance from stacking position, radians) to randomize new stacking position. Default is 5 beams
- **upperLimit** – Upper spatial limit (distance from stacking position, radians) to randomize new stacking position. Default is 10 beams
- **save** – either 'central_pixel' or 'all'. If 'central_pixel' only the spectrum of central pixel is saved.
- **spectralMethod** – Method to select the central frequency of the stack. See LineStacker.image.stack.
- **seed** – see the module description ('thread' is the default backend here: the cubes stay shared)
- **n_jobs** – see the module description ('thread' is the default backend here: the cubes stay shared)
- **backend** – see the module description ('thread' is the default backend here: the cubes stay shared)
- **verbose** – print what is done and a progress bar (default: LineStacker.verbose)
- ****kwargs** – weighting, maskradius, regridFromZ... passed to LineStacker.image.stack's preparation

Returns

list – [standard deviation, mean, all random stacks].

LineStacker.analysisTools.**randomizeCoords**(*coords*, *beam=0*, *lowerLimit='default'*, *upperLimit='default'*, *imagenames=None*, *hdul='auto'*, *seed=None*)

New positions at a random distance (uniform between lowerLimit and upperLimit) and in a random direction from each coordinate.

Parameters

- **coords** – A list of LineStacker.Coord coordinates
- **lowerLimit** – Lower limit of the distance (radians) to the original position.

  Default is 5 beams
- **upperLimit** – Upper limit of the distance (radians) to the original position.

  Default is 10 beams
- **beam** – beam size (radians), needed if lowerLimit or upperLimit are set to 'default'
- **imagenames** – needed for pixel coordinates (to convert distances to pixels)

LineStacker.analysisTools.**regridFromZ**(*coords*, *imagenames=None*, *stampsize=32*, *N_chans=0*, *fEm=0*, *writeImage=True*, *regridMethod='scaleToMin'*, *hdul='auto'*, *outdir='./regridedImages'*, *verbose=None*)

Cube regridding function correcting the observed linewidth change due to redshift. Spectral dimensions are rebinned to a new_size=old_size*zRatio where zRatio=(1+z)/(1+zRef) where z is the observed line redshift and zRef is either the smaller (**scaleToMin**) or bigger (**scaleToMax**) redshift among the studied lines. Spectra are resampled with a cubic spline through the valid channels (separately between blank channels, which are never used): new channels between a valid and a blank channel, or beyond the last channel, are NaN.

LineStacker.image.stack(. . . , regridFromZ=True) corrects the line widths in memory instead, without writing cubes (with linear interpolation).

Parameters

- **coords** – A list of LineStacker.Coord of all target coordinates.
- **imagenames** – Name of images to extract flux from.
- **stampsize** – size of target image in pixels
- **N_chans** – unused, kept for compatibility
- **fEm** – rest emission frequency of the line (frequency cubes only)
- **writeImage** – boolean, if set to **True** regridded images will be written to disk (in **outdir**)
- **regridMethod** – Method of regridding, can be set either to **'scaleToMin'** or **'scaleToMax'**. In the first case all spectra are regridded to match the smallest redshift (overgridding), all spectra are regridded to match the highest redshift in the other case (undergridding).
- **outdir** – folder for the regridded cubes (default ./regridedImages; emptied first)
- **verbose** – print where the regridded cubes are written, and a progress bar (default: LineStacker.verbose)

Returns

tuple or list – If writeImage is **True**: (imagenames, coords), the regridded cubes and their coordinates. Stack them as they are, with LineStacker.image.stack(coords, imagenames=imagenames, spectralMethod='z', fEm=fEm): a regridded cube's spectral axis is labelled in the rest frame (its coordinate has z = 0, channels of rest-frame width), and sources already at the reference redshift keep their original cube and redshift. Regridded cubes are stamps of stampsize pixels around each source, with the sky coordinates of the original cube. If writeImage is **False**: the list of regridded stamps.

LineStacker.analysisTools.**regridFromZ1D**(*spectra*, *regridMethod='scaleToMin'*, *verbose=None*)

1D regridding function correcting the observed linewidth change due to redshift. Spectra are rebinned to a new_size=old_size*zRatio where zRatio=(1+z)/(1+zRef) where z is the observed line redshift and zRef is either the smaller (**scaleToMin**) or bigger (**scaleToMax**) redshift among the studied lines. Fluxes are resampled with a cubic spline through the valid channels (separately between blank channels, which are never used): new channels between a valid and a blank channel, or beyond the last channel, are NaN. Spectral axes, per-channel weights and line centres (centerIndex) follow the new channels.

Parameters

- **spectra** – List of spectra (LineStacker.OneD_Stacker.Spectrum objects).
- **regridMethod** – Method of regridding, can be set either to **'scaleToMin'** or **'scaleToMax'**. In the first case all spectra are regridded to match the smallest redshift (overgridding), all spectra are regridded to match the highest redshift in the other case (undergridding).
- **verbose** – print what is done and a progress bar (default: LineStacker.verbose)

Returns

list of LineStacker.OneD_Stacker.Spectrum – the regridded spectra (copies).

9.5 LineStacker.tools

LineStacker.tools.**ProgressBar**(*n*, *total*, *verbose=None*)

Draws a progress bar (at most about 100 redraws).

Parameters

- **n** – current step
- **total** – total number of steps
- **verbose** – draw the bar (default: LineStacker.verbose)

LineStacker.tools.**resolve_n_jobs**(*n_jobs*)

Number of workers for an n_jobs argument (None/1 = serial, -1 = all cores).

9.6 LineStacker.tools.fit

Gaussian fitting of spectra (single, double, and symmetric double peaks).

The fitting functions ignore non-finite samples (NaN channels) and raise *FitError* (a RuntimeError) when a fit cannot be done or does not converge: too few valid samples, a constant spectrum, invalid initial values, or non-finite fitted parameters.

exception LineStacker.tools.fit.**FitError**

A fit could not be done or did not converge.

LineStacker.tools.fit.**Gauss**(*f=None*, *f0=0*, *dV='default'*, *emittedF=0*, *sPeak=1.0*, *dF=1*, *noise=False*, *sigmaNoise=0*)

Gaussian line on the axis f: peak sPeak at f0 and FWHM dF; with dF=0 the FWHM is that of the velocity width dV (m/s) at the frequency emittedF, dF = dV emittedF / c. noise=True adds Gaussian noise of standard deviation sigmaNoise.

LineStacker.tools.fit.**gaussFct**(*x*, *a*, *x0*, *sigma*, *constant=0*)

Gaussian a exp(-(x-x0)**2 / (2 sigma**2)) + constant.

LineStacker.tools.fit.**DoubleGauss**(*f=None*, *amp1=0*, *amp2=0*, *f01=0*, *f02=0*, *sigma1=0*, *sigma2=0*)

Sum of two Gaussians (amplitudes amp1, amp2, centres f01, f02, widths sigma1, sigma2).

LineStacker.tools.fit.**DoubleGaussPeak**(*f=None*, *amp=0*, *dist=0*, *sigma=0*)

Two Gaussians of the same amplitude and width, dist apart, centred on the middle of the axis f.

LineStacker.tools.fit.**GaussFit**(*fctToFit=None*, *fullFreq=None*, *sigma='default'*, *returnInfos=False*, *returnError=False*)

Simple Gaussian fitting function. Return order: fitting function, (fitting parameters), (error on fitting parameters). Non-finite samples are ignored; raises FitError (a RuntimeError) if the fit fails.

Parameters

- **fctToFit** – One dimensional array to be fitted
- **fullFreq** – Spectral dimension array. **Not required**
- **sigma** – initial width for fitting, **'default'** is 3 bins
- **returnInfos** – If set to **True** function returns fitted parameters
- **returnError** – If set to **True** function returns error on fitted parameters

LineStacker.tools.fit.**DoubleGaussFit**(*fctToFit=None*, *fullFreq=None*, *sigma1='default'*, *sigma2='default'*, *ampScale=0.1*, *returnAllComp=False*, *returnInfos=False*, *returnError=False*)

Double Gaussian (sum of two Gaussians) fitting function. Return order: fitting function, (first Gaussian, second Gaussian), (fitting parameters), (error on fitting parameters). Non-finite samples are ignored; raises FitError (a RuntimeError) if the fit fails.

Parameters

- **fctToFit** – One dimensional array to be fitted.
- **fullFreq** – Spectral dimension array. **Not required**
- **sigma1** – Initial width of the first Gaussian component for fitting, **'default'** is 3 bins.
- **sigma2** – Initial width of the second Gaussian component for fitting, **'default'** is 6 bins.
- **ampScale** – Initial amp ratio (between the two Gaussian components) for fitting, **default** is 0.1.
- **returnAllComp** – If set to **True** function returns both individual components.
- **returnInfos** – If set to **True** function returns fitted parameters.
- **returnError** – If set to **True** function returns error on fitted parameters.

LineStacker.tools.fit.**DoublePeakFit**(*fctToFit=None*, *fullFreq=None*, *returnInfos=False*)

Fit of two Gaussians of the same amplitude and width placed symmetrically about the middle of the spectral axis (see DoubleGaussPeak). Returns the model (and the parameters amp, dist, sigma with returnInfos=True). Raises FitError if the fit fails.

9.7 LineStacker.tools.parallel

Small helper to spread Monte-Carlo iterations (bootstraps, subsamples, random stacks) over several CPU cores.

Every function in LineStacker that accepts `n_jobs` draws all its random numbers up front, in the calling process, and only distributes the deterministic work. Results are therefore identical whatever `n_jobs` or `backend` is, and reproducible with `seed`.

`backend='process'` (default) runs workers in separate processes and is the right choice for Python-heavy work such as Gaussian fits. `backend='thread'` avoids starting processes and copying data; it helps when the work is mostly large numpy operations (e.g. median stacks of big cubes).

When using processes from a script on macOS or Windows, protect the script's entry point:

```
if __name__ == '__main__':
    main()
```

otherwise each worker re-runs the script while starting.

LineStacker.tools.parallel.**set_default_backend**(*backend*)

Set the backend used when a function is called with backend=None ('process' or 'thread').

LineStacker.tools.parallel.**get_shared**(*name*)

Access, from a worker function, an object passed through `shared`.

LineStacker.tools.parallel.**map_chunks**(*func*, *tasks*, *n_jobs=1*, *backend=None*, *shared=None*, *progress=True*)

Apply `func` to every element of `tasks` and return the results in order. `func` must be a module-level function (so it can be sent to worker processes). Large inputs common to all tasks go in `shared` (a dict); workers read them with *get_shared()*.

Parameters

- **func** – function taking one task and returning its result
- **tasks** – list of tasks (usually slices of iterations)

- **n_jobs** – number of workers; 1 runs serially, -1 uses all cores
- **backend** – ‘process’ or ‘thread’ (None uses the default, see set_default_backend)
- **shared** – dict of read-only objects made available to func via get_shared
- **progress** – show LineStacker’s progress bar

CHAPTER

TEN

LINESTACKER 2.0: PYTHON 3 / CASA 6 PORT

Tested with Python 3.13 (requires >= 3.9), NumPy 2.5, SciPy 1.18, astropy 8.0 and CASA 6.7.6 (casatools from PyPI), on the example data in `LineStacker/Example` and on the synthetic test suite in `tests/` (183 tests; the 12 that need casatools are skipped without it).

10.1 Installing

```
pip install -e .[casa]        # with CASA 6 for CASA images
pip install -e .              # FITS cubes only
python -m pytest tests        # optional: run the tests
```

To use LineStacker from the CASA prompt (monolithic CASA tarball), install its dependencies with CASA's own pip, so they match CASA's NumPy:

```
PYTHONNOUSERSITE=1 ~/casa-6.7.6-14-py3.12.el9/bin/pip3 install 'astropy>=7.
↪2'
```

Mixing CASA with packages from another environment (e.g. a venv on PYTHONPATH) breaks when versions disagree: CASA 6.7.6 bundles NumPy >= 2.4, which astropy < 7.2 cannot use (`module 'numpy' has no attribute 'in1d'`). `import LineStacker` warns when it detects this combination.

CASA is only imported to read or write CASA images (FITS-only use never starts CASA). When LineStacker starts it, CASA's log goes to one file, `<temporary dir>/linestacker-casa-<user>.log` (appended to), and CASA's messages go there instead of the terminal, so no `casa-<date>.log` piles up in the working directory. `LineStacker.casa_logfile` changes this (a path, `None` for no log, `'casa'` for CASA's default); a `logfile` set in `~/.casa/config.py` is used, and inside a CASA session (or after importing casatools/casatasks yourself) CASA's own settings apply. CASA 5 (Python 2) is no longer supported.

10.2 One stacking module: `LineStacker.image`

`line_image` and the old continuum module `image` are merged into a single `LineStacker.image`, which does what `line_image` did, plus continuum stacking with the same functions:

```
LineStacker.image.stack(coords, imagenames=names, fEm=fEm)      # line␣
↪stack
LineStacker.image.stack(coords, imagenames=names, continuum=True)
```

- Images without a spectral axis (or with one channel) are stacked as continuum automatically, without warning.
- `continuum=True` on cubes with a spectral axis issues a warning and stacks a collapsed version of each cube, made in memory (`collapse='sum'` default, or `'mean'`, `'median'`; blanked channels are ignored). The output header records it in `COLLAPSE`. Only the collapsed arrays are kept, not the full cubes.
- Coordinates without a spectral value (`readCoords(..., lineON=False)`) also select continuum stacking (same warning if cubes get collapsed).

- Line stacking with an image that has no spectral axis raises an error pointing to `continuum=True`.
- Continuum stacks return the same list as line stacks (the "spectrum" has one channel); FITS output is 2D when the inputs had no spectral axis.
- From the old continuum module: weightings `'sigma'` ($1/\sigma$, mask of two beams by default) and `'pb'` (squared primary-beam response) are available to all stacks; `getFlux` is kept. `primarybeam=` now takes the primary-beam image of each cube (FITS or CASA, e.g. tclean's `.pb`; one path, a list in the order of `imagenames`, or a dict `{cube: pb}`), read at each source's position and line channel, or a function `f(dx, dy, nu)` such as `LineStacker.image.GaussianPrimaryBeam(dish=12.)` (FWHM 1.13 λ/D, as LineStacker.uv). The `LineStacker.pb` models are gone (see below). Its `noise` and `boot_straping` are covered by `analysisTools.stack_estimator` and `bootStraping_Cube`, which accept `continuum=True`. Dropped: `psfmode='star'`, `calculate_peak_fluxes` and `calculate_flux_weights` (`.flux` images).
- `import LineStacker.line_image` no longer works: use `LineStacker.image` (same `stack` arguments).

10.3 What the stacked cube contains

- Stamps are cut on each cube's own pixel grid, centred on the source; cubes are not reprojected. The output keeps the first cube's pixel size and orientation (a rotated grid is written with a PC matrix).
- All cubes must have the same kind of spectral axis (FREQ, WAVE, AWAV, VRAD...); stacking frequency with wavelength cubes, or using a non-linear axis (`WAVE-LOG`, `FREQ-TAB...`) in a line stack (any `spectralMethod`), is an error. Units are converted to the first cube's (Hz and GHz cubes stack together); `fEm`, `lEm` and observed values are in that unit. Different `SPECSYS` frames, or `SPECSYS` missing in some cubes, give a warning: LineStacker does not convert frames.
- Pixel units (`BUNIT`) are compared: scaled units are converted to the first cube's (mJy/beam to Jy/beam), recognised but physically different units (K and Jy/beam, Jy/pixel and Jy/beam) are an error, and cubes without `BUNIT`, or with units astropy cannot read, give a warning and are stacked without conversion.
- Cubes on a different grid from the first one (pixel size, orientation, projection, celestial frame, distortions) give a warning; stamps are not reprojected.
- BMAJ, BMIN are the mean of the input beams and BPA their axial mean (179° and 1° give 0°), an approximation of the stack's beam.
- Weights must be finite and >= 0 (a negative or NaN weight is an error). Every weighting multiplies the coordinates' own weights.

10.4 1D stacking: `Image` is now `Spectrum`

The object holding one spectrum to stack is `LineStacker.OneD_Stacker.Spectrum` (was `Image`), with the same arguments and attributes. The 1D functions take `spectra` instead of `Images`/`images`: `Stack(spectra, ...)`, `bootstraping_OneD(spectra, ...)`, `subsample_OneD(spectra, ...)`, `rebin_OneD(spectra)`, `regridFromZ1D(spectra, ...)`. Positional calls are unchanged; calls passing `Images=` by keyword need updating.

Velocities are in km/s. A spectrum with velocities and a redshift (`Spectrum(spectrum=..., z=z, fEmLine=f0)`) has velocities relative to the rest frequency `f0` (as a CASA spectrum with `restfreq=f0`); its line is at v = c z/(1+z) in the radio convention (default, as CASA and LineStacker.uv), c z with `velConvention='optical'`, or the relativistic velocity with `'relativistic'`. `velocity_to_frequency` and `frequency_to_velocity` do the conversions.

10.5 Stacking positions: catalogue, arrays or coordinate files

- `LineStacker.readCatalog(catalog)`: one table for the whole sample, one row per source: `ra`, `dec`, `z` (optional; without it the positions are for continuum stacking), `weight` (optional) and `image`,

the cube of each source. Reads CSV, ECSV, FITS tables, VOTable (any astropy table format), an astropy `Table`, a pandas `DataFrame` or a dict. Column names are matched case-insensitively, common alternatives are recognised (`RAJ2000`, `DEJ2000`, `redshift`, `cube`, ...), and any name can be given (`readCatalog(cat, ra='alpha', z='zspec', image='file')`). Positions in degrees (or `unit='rad'`), with units from the table, or sexagesimal (RA in hours, CASA's dotted declinations accepted); each value is read on its own, so a column mixing forms is read correctly. Relative cube names are looked up next to the catalogue, then from the working directory (`imagedir=` overrides).

- `LineStacker.makeCoords(ra, dec, z=..., weight=..., image=...)`: the same from lists, numpy arrays, astropy quantities or a `SkyCoord` (notebooks).
- The cube of a source is never guessed: it comes from the catalogue/`image=`, or, when no cube is named, from a single cube given as `imagenames`. A cube missing from `imagenames` is an error. `imagenames` can be omitted when every source names its cube.
- `readCoords` (one file per cube) works as before.
- `LineStacker.image.readCube(name)` reads a FITS file or CASA image as a numpy array (x, y, stokes, channel), for plots and checks.
- `rebin_CubesSpectra(..., outdir=)` writes the rebinned cubes to a folder.

10.6 Examples notebook

`LineStacker/Example/LineStacker_examples.ipynb` runs every example (1D stacking, bootstrapping and subsampling; cube stacking from coordinate files, a catalogue and arrays; line fits, rebinning and bootstrapping; continuum stacking and random positions) on the example data in about 30 s, writing to `Example/notebook_outputs/`. It is saved with its outputs (figures).

10.7 Messages: `verbose`

`LineStacker.verbose` (default `True`) makes LineStacker report what it does: positions read, the full path of every file written (stacked cube, weights, sub-cubes, rebinned or regridded cubes) and progress bars. Each function that prints takes `verbose=True/False` to override it for one call. With verbose off LineStacker prints nothing; problems (zero-noise stamps, collapsed cubes, failed fits...) are Python warnings and show either way.

10.8 What works now that did not

- **CASA images as input.** `line_image` had been rewritten to read FITS only, so the shipped examples (CASA `.image` cubes) failed with `IsADirectoryError`. All modules now share one reader (`LineStacker/_io.py`) that takes FITS files or CASA images (exported through casatools; both kinds can be mixed in one stack), in any axis order (RA/Dec swapped, Stokes before or after the spectral axis, no Stokes, 2D images), with `CDELT` or `CD` matrices and frequency or wavelength (`FREQ`, `WAVE`, `AWAV`) axes. Cubes are read once and cached (`LineStacker._io.clear_cache()` frees them).
- **CASA 6 tools.** The shim imported `casalog` from `casatools`, which CASA 6 does not export, so it failed on every real CASA 6 installation; its CASA 5 fallback imported itself. Both fixed and checked against casatools 6.7.6.
- **Output cubes.** `outfile='x.fits'` writes FITS, any other name (e.g. `stackResult.image`) a CASA image with axes RA, Dec, Stokes, Frequency as before. The header is rebuilt cleanly instead of averaging every keyword of every input header.
- **NumPy 2.** `np.NaN`, `np.float`, `np.cast`, `np.rank`, list indexing and int-array in-place float maths were removed or broke `OneD_Stacker.Stack`, `congrid`, `regridFromZ1D`, `rebin_OneD` and spline regridding. Comparing a weights array with `'1/A'` raised "truth value is ambiguous".
- **Modules that used the removed Stacker `CoordList` model** (`image` (continuum stacking), `bootStraping_Cube`, `stack_estimator`, `rebin_CubesSpectra`, cube `regridFromZ`, and

`randomCoords` / `randomizeCoords`, which only printed 'TODO') are ported to `Coord(ra, dec, spec)`. `Coord` also answers to `x`, `y`, `z` for older scripts.

- **regridFromZ** in `line_image.stack` was silently ignored; it now resamples each source to a common rest-frame channel (linear interpolation, fractional line centres), for frequency and wavelength cubes.
- `readCoords`: CASA `-027.40.59.866` declinations, blank/comment lines, delimiter fallback (comma, space, tab), `type='pixel'` (was ignored). Coordinate files are matched to `imagenames` by name, or else by file order.
- **Gaussian fits** (`LineStacker.tools.fit`) ignore NaN channels and raise `FitError` (a `RuntimeError`) for too few points, constant data or a fit that does not converge; the bootstraps and grades turn those into NaN or 0. `DoubleGaussPeak` is centred on the middle of its axis (it assumed an axis starting at 0).
- **Input checks** with clear messages: empty spectrum or image lists, negative or non-integer `nRandom`, `minSize`/`maxSize` of `subsample_OneD`, `regridMethod`, rebinning widths, `randomCoords` margins, empty cube names and non-finite `z`/weights in catalogues, equatorial frames only (`makeCoords(SkyCoord)` converts e.g. Galactic positions). Coordinates in different frames (ICRS, FK5…) are each converted.
- **Cached cubes** are read again when their file changes (for CASA images, any file of the image directory); the stale copy is dropped.
- Smaller crashes: `KeyError: 'freqlen'` when a line was near the upper edge of a cube, `coord.z` in obsSpec mode, `returnWeights`, `plotIt`, `stampsize=1` in `stack_estimator`, non-square images in `noise_estimator`, float slice indices in `rebin_CubesSpectra`, `freqBin` never rescaled when rebinning 1D spectra, `LineStacker.tools.GaussFit` typo in the 1D bootstrap.

10.9 Changes in results (please check against your expectations)

1. **Stack centre moves by one channel** for FITS input. The FITS rewrite used the 1-based `CRPIX3` where the CASA code used CASA's 0-based reference pixel. The new code agrees with CASA's own `topixel`.
2. Line centres are rounded half away from zero, as Python 2's `round()` did (Python 3 rounds halves to even).
3. Median stacks ignore channels that fall outside a source's cube instead of counting them as zeros. Mean stacks already gave those channels zero weight.
4. `bootStraping_Cube` 'amp'/'ampAndWidth'/'outflow' now use the spectrum summed over the stamp, as documented. The code summed axes (1,2,3), i.e. a spatial profile.
5. Weights are computed on copies: the user's `Coord` objects are not modified, and `weighting='sigma2'` no longer compounds when the same coordinates are stacked twice.
6. Cubes whose spectral axis runs the other way (positive vs negative CDELT3) are flipped to match the first cube instead of being stacked mirrored.
7. Channels whose total weight is zero give NaN (blank) instead of ZeroDivisionError; sigma2F channels with zero noise get weight 0 instead of infinity.
8. Gaussian fits that do not converge are stored as `[nan, nan]` (with a warning) instead of aborting a whole bootstrap.
9. `hdul` defaults to `'auto'` (first HDU with an image: 0 for ALMA/CASA FITS, 1 for MUSE). An explicit `hdul=1` still works.
10. `noise_estimator(maskCenter=[x, y])` uses x along RA and y along Dec as documented; it no longer blocks on `plt.show()` (`plotIt=False` disables the plot).
11. `maximizeSNR` (the default grade of `subsample_OneD`) returns 0 when the Gaussian fit converges on a line centred outside the spectrum (it can lock onto a stack edge). Its noise was then computed from a negative slice index, i.e. a few channels at the end, giving grades of ~1000 instead of ~4 that dominated

the summed grades: the subsampling example no longer singled out the bright spectra. With the fix it does again (with 10 000 iterations already). The fit's sigma is also used as an absolute value.

12. Random draws are vectorised, so a given `np.random.seed` gives different (statistically equivalent) samples than LineStacker 1.x. All Monte-Carlo tools also take `seed=`.

13. `weighting='sigma2F'` multiplies the coordinates' weights by $1/\sigma^2$ (it replaced them, so catalogue weights were lost), and σ of each channel uses only the valid pixels outside the mask (masked and blank pixels were counted as zeros, which lowered σ). Unchanged for unit weights, no mask and no blank pixels, as in the examples.

14. `weighting='1/A'` gives weight 0 (with a warning) to sources whose amplitude is not positive, instead of a negative or infinite weight.

15. Weighted 1D stacks: channels where a spectrum's flux is NaN no longer count in the sum of weights (they pulled the mean towards 0).

16. Spectra given as amplitudes only, with an odd number of channels, lost their last channel in 'full' stacks (Python 3 rounding of the default velocity axis).

17. `Spectrum(z=..., fEmLine=...)` with velocities is centred on the line (`velToFreq` returned `fObs v/(v+c)`, not a frequency).

18. `regridFromZ` (writing cubes): regridded cubes have a rest-frame spectral axis (CDELT3 was the observed width, so the stack's CDELT3 was wrong) and keep the sky coordinates of the original cube (CRVAL was set to 0, 0). Its result is stacked as returned, with `spectralMethod='z'` and `fEm`.

19. Beam position angles are averaged as axes (BPA changes only when the input angles straddle $\pm 90°$).

20. Cubes in compatible but different intensity units (mJy/beam, Jy/beam) are converted to the first cube's unit before stacking (they were averaged as raw numbers).

21. `regridFromZ`: the coordinates returned for regridded cubes are the sources' own (fractional) pixels in the stamps, not the integer stamp centre, so their sky positions are exact.

22. 1D stacks leave out infinite fluxes, as they do NaN (one infinite channel made the stack infinite).

23. **Blank pixels stay NaN** (they were set to 0, as in LineStacker 1.x). Stacks leave them out pixel by pixel; stacked pixels without valid data, and stack channels that no source covers, are NaN instead of 0; pixels that are exactly 0 now count as data in the noise-based weightings and `noise_estimator`; collapsed images keep NaN where every channel is blank. `readCube` returns NaN for blank pixels. A blank channel blanks only the stack channels interpolated from it (a stack channel that falls exactly on a valid channel keeps its value). Rebinning and regridding never use blank channels: with linear interpolation (`rebin_OneD`, `rebin_CubesSpectra`) a new channel interpolated from a blank one is NaN; the cubic spline of `regridFromZ1D` and `regridFromZ` runs through each stretch of valid channels separately, so only new channels between a valid and a blank channel are NaN. Blanks thus stay local instead of spreading over the whole spectrum. The `maximizeSNR` grade measures the noise on valid channels only. 1D stacks also give NaN, instead of 0, for channels without data (no spectrum covers them, or only blank fluxes or zero weights), as cube stacks do.

24. **Regridding to a common rest-frame channel width** (`regridFromZ1D`, `Stack(regridFromZ=True)`, `regridFromZ`) samples old channel j (1+z_ref)/(1+z) for new channel j, so every source gets exactly the reference channel width (the number of channels was rounded and the width followed it, up to half a channel over the spectrum). New channels beyond the last old channel are NaN; the spline used to give 0 there, and a velocity of 0 at the end of the axis. Spectral axes are interpolated linearly (the spline bent them near the ends), and the channel widths, line centres (rounded, they were truncated), per-channel weights and `spectrum` arrays follow the new channels. Regridded cubes label their spectral axis with the exact, fractional, channel of the line (it was rounded to a whole channel).

25. `rebin_OneD` moves the line centre (`centerIndex`) with the rebinned channels (it kept the old channel, so line-centre stacks of rebinned spectra were misaligned unless `fit=True`), and compares line widths in channels (fitted widths in km/s mixed spectra with different channel widths).

26. The spectral axis of a 1D `Spectrum` must be evenly spaced (within 5% of a channel, so rounded values from text files are fine): every channel is located with one channel width, which is now the

mean spacing (it was the first spacing, which rounding of the values could make off by a few percent). `weights='sigma2'` with `fit=True` left the line in the noise estimate of spectra with decreasing axes. A line centre outside its spectrum is now an error when stacking (it made very long stacks).

On the example data (no blank pixels) all stacks, with every weighting, line and continuum, are bit-identical to the first 2.0 port; items 13-26 change results only for data that hit those cases (the examples do not regrid spectra by redshift).

10.10 Speed and parallelism

The 1D stacker, the bootstraps and the random-position estimator were vectorised; this matters more than extra cores. Times on the example data, 2-core sandbox (expect better scaling with more cores):

task	LineStacker 1.x	2.0 serial	2.0, 2 cores
1D median bootstrap, 50 spectra, 10 000 draws	263 s	1.6 s	0.9 s (threads)
1D mean bootstrap, 100 000 draws	~45 min (est.)	0.8 s	
1D bootstrap with Gaussian fits, 20 000 draws		18.0 s	10.9 s (processes)
1D subsample (maximizeSNR), 10 000 draws	~190 s (est.)	13.7 s	8.2 s (processes)
cube stack, 50 CASA cubes (first / cached)		2.4 s / 0.09 s	
continuum stack of the same cubes (collapsed in memory)		1.6 s / 0.16 s	
cube mean bootstrap, 10 000 draws	broken	0.2 s	
cube median bootstrap, 1 000 draws	broken	8.5 s	4.3 s (threads)
cube bootstrap with fits, 20 000 draws	broken	11.7 s	6.9 s (processes)
stack_estimator, 2 000 random stacks	broken	0.2 s	

`bootstraping_OneD`, `bootStraping_Cube`, `subsample_OneD` and `stack_estimator` take `n_jobs` (-1 = all cores), `backend` (`'process'` or `'thread'`; a sensible one is chosen per task) and `seed`. Random numbers are drawn before the work is split, so results are identical for any `n_jobs`. Worker processes start with `spawn` (safe with CASA and on macOS), which costs ~0.7 s, so processes pay off for runs longer than a few seconds. On macOS/Windows, scripts using processes need an `if __name__ == '__main__':` guard (see the examples).

10.11 Removed

- `LineStacker.modsub` and `LineStacker.pb`: the uv-domain code of Stacker (it needed the C++ `libstacker`). uv-domain stacking is `LineStacker.uv` (2.1); primary-beam weighting of image stacks takes primary-beam images or a function (see above).
- The vendored `interval` package (unused).
- CASA 5 support (`taskinit`): CASA 5 runs Python 2.
- Helpers working on the module state of the last stack: `image.calculate_sigma2_weights`, `calculate_sigma2_weights_spectral`, `calculate_amp_weights`, `myVeryOwnWeightedAverage` and the module variables (`image.data`, `skymap`, `unusedFrequencies...`), which kept every cube of the last stack in memory; use `weighting=` and the values `stack` returns. Also `analysisTools.randomizeSample` and `Coord.setWeightToX`/`setZeroWeightLeft`/`setZeroWeightRight` (unused).
- The documentation inside the package (`LineStacker/docs`): it is built from `docs/source` into `docs/` (the github.io site).

10.12 Review fixes (October 2026)

A code review of the 2.0 port led to: the weighting fixes (items 13-15 above), the spectral-unit and axis checks, grid warnings and rotated output WCS, primary-beam offsets on rotated grids (the cross terms of the pixel matrix were ignored), the velocity conversion, `regridFromZ`'s cube labels, NaN-aware fits, input validation, cache invalidation of modified CASA images, closing every CASA tool, and per-call isolation of the inputs of parallel jobs (two jobs in different threads could see each other's data). Each fix has a test in `tests/test_weights_wcs.py` or `tests/test_validation.py`.

A second review led to: non-linear spectral axes refused in every line mode (the stack's header could not describe them), intensity-unit checks and conversion, projection/frame/distortion checks, fractional pixels from `regridFromZ`, infinite fluxes and one-channel spectra in 1D, `FitError` for every fit failure (`DoublePeakFit` on short inputs, default widths from valid channels only), the same "at least 2 valid pixels" rule for `'sigma'` and `'sigma2'` as for `'sigma2F'`, integer subsample sizes, one image path and limit checks in both `randomizeCoords`, `congrid(minusone=True)` with a target size of 1, `None`/NaN cube names in catalogues, primary-beam cubes on another spectral axis (an error), `LineStacker.cache_check` to trust the cube cache, a lock around the thread variables set for worker processes, and subprocess tests of CASA's log settings. Tests: `tests/test_review2.py`.

A third round: blank pixels stay NaN (see "Changes in results", 23), SIP and lookup-table distortions are compared term by term, the Gaussian primary beam uses spherical sky offsets, `randomCoords(ncoords)` and `randomizeCoords(maxTries)` must be integers, `noise_estimator` and `stack_estimator` need `nRandom >= 1`, default fit widths use the median channel spacing, and spectral axes of spectra must be finite and steadily increasing or decreasing.

A fourth round finished the NaN handling: exact channels next to a blank one are no longer blanked (`NaN * 0` is NaN) when stamps are cut, rebinning and regridding share one NaN-aware resampling (blank channels used by an interpolation blank its result, and nothing else), `rebin_CubesSpectra(regionSize=...)` keeps channels blank in the whole region as NaN for its width fits, `maximizeSNR` ignores blank channels in its noise (and gives 0 with fewer than 2 valid ones), spectra with a one-element velocity or frequency axis raise `ValueError`, and distortion signatures with different terms compare as different. `regridFromZ` with blanks now also works when it upsamples (it raised an error). Tests: `tests/test_review4.py`.

A fifth round and a release review: splines that never use blank channels (the blank mask approximated the spline's reach, which its prefilter extends over the whole spectrum), evenly spaced 1D axes, `sigma2` weights of fitted spectra with decreasing axes, NaN for 1D channels without data, and the resampling fixes of items 24-25 (axes, centres, widths and weights of rebinned and regridded spectra; exact channel widths and line labels of regridded cubes). Also: line centres outside a spectrum, `center='zero_vel'` without velocities, Gaussian-fit centring of frequency spectra (it used the velocities), AIPS `FELO` axes treated as non-linear, `rebin_CubesSpectra(regionSize=...)` regions outside the cube (and of the size asked for: an odd size gave one pixel less, the region is now placed as the stacker places stamps), a warning and weight 0 for blank primary-beam responses (beyond the primary-beam cutoff, also for single-plane primary-beam images, which gave a silent 0), dimension checks in the NaN-aware resampling, NaN-safe `maximizeAmp` and 1D bootstrap amplitudes, and the missing docstrings of `congrid`, `maximizeAmp` and `maximizeOutflow`. Tests: `tests/test_review5.py`.

The version stays 2.0.0 until the first 2.0 release; if a 2.0.0 was already distributed, release this as 2.0.1.

10.13 Example scripts

`LineStacker/Example/*/*.py` are ported (the originals were Python 2 and used `taskinit`); `cube/continuum_example.py` is new. They read cubes with `LineStacker.image.readCube` (no casatools calls of their own). The cube scripts accept the data folder as an argument instead of the file dialog: `python basic_example.py data`. They use `str.removesuffix('_coord.txt')`; the old `strip('_coords.txt')` removed characters, not a suffix, and only worked by luck with the example names.

CHAPTER

ELEVEN

CITING LINESTACKER

If LineStacker is useful for your work, please cite Jolly, Knudsen & Stanley (2020), MNRAS 499, 3992. LineStacker started from Stacker (Lindroos et al. 2015, MNRAS 446, 3502).

The code is on GitHub, under the GNU General Public License.

CHAPTER

TWELVE

INDICES

- genindex
- modindex
- search

PYTHON MODULE INDEX

L

LineStacker, 33
LineStacker.analysisTools, 41
LineStacker.image, 36
LineStacker.OneD_Stacker, 39
LineStacker.tools, 48
LineStacker.tools.fit, 48
LineStacker.tools.parallel, 49

INDEX

B

bootStraping_Cube() (*in module LineStacker.analysisTools*), 42
bootstraping_OneD() (*in module LineStacker.analysisTools*), 41

C

cache_check (*in module LineStacker*), 33
casa_logfile (*in module LineStacker*), 33
clear_cache() (*in module LineStacker.image*), 39
congrid() (*in module LineStacker.analysisTools*), 43
Coord (*class in LineStacker*), 33
copy() (*LineStacker.Coord method*), 34

D

DoubleGauss() (*in module LineStacker.tools.fit*), 48
DoubleGaussFit() (*in module LineStacker.tools.fit*), 48
DoubleGaussPeak() (*in module LineStacker.tools.fit*), 48
DoublePeakFit() (*in module LineStacker.tools.fit*), 49

F

FitError, 48
freqToVel() (*LineStacker.OneD_Stacker.Spectrum method*), 40
frequency_to_velocity() (*in module LineStacker.OneD_Stacker*), 40
fwhm() (*LineStacker.image.GaussianPrimaryBeam method*), 39

G

Gauss() (*in module LineStacker.tools.fit*), 48
gaussFct() (*in module LineStacker.tools.fit*), 48
GaussFit() (*in module LineStacker.tools.fit*), 48
GaussianPrimaryBeam (*class in LineStacker.image*), 39
get_shared() (*in module LineStacker.tools.parallel*), 49
getFlux() (*in module LineStacker.image*), 39
getPixelCoords() (*in module LineStacker*), 36

L

LineStacker
    module, 33
LineStacker.analysisTools
    module, 41
LineStacker.image
    module, 36
LineStacker.OneD_Stacker
    module, 39
LineStacker.tools
    module, 48
LineStacker.tools.fit
    module, 48
LineStacker.tools.parallel
    module, 49

M

makeCoords() (*in module LineStacker*), 34
map_chunks() (*in module LineStacker.tools.parallel*), 49
maximizeAmp() (*in module LineStacker.analysisTools*), 44
maximizeOutflow() (*in module LineStacker.analysisTools*), 44
maximizeSNR() (*in module LineStacker.analysisTools*), 44
module
    LineStacker, 33
    LineStacker.analysisTools, 41
    LineStacker.image, 36
    LineStacker.OneD_Stacker, 39
    LineStacker.tools, 48
    LineStacker.tools.fit, 48
    LineStacker.tools.parallel, 49

N

noise_estimator() (*in module LineStacker.analysisTools*), 45

P

ProgressBar() (*in module LineStacker.tools*), 48

R

randomCoords() (*in module LineStacker*), 36
randomizeCoords() (*in module LineStacker*), 36
randomizeCoords() (*in module LineStacker.analysisTools*), 46
readCatalog() (*in module LineStacker*), 35
readCoords() (*in module LineStacker*), 34

readCoordsNamesGUI() (*in module LineStacker*), 34
readCube() (*in module LineStacker.image*), 39
rebin_CubesSpectra() (*in module LineStacker.analysisTools*), 44
rebin_OneD() (*in module LineStacker.analysisTools*), 43
regridFromZ() (*in module LineStacker.analysisTools*), 46
regridFromZ1D() (*in module LineStacker.analysisTools*), 47
resolve_n_jobs() (*in module LineStacker.tools*), 48
round_half_up() (*in module LineStacker*), 33

S

set_default_backend() (*in module LineStacker.tools.parallel*), 49
Spectrum (*class in LineStacker.OneD_Stacker*), 39
stack() (*in module LineStacker.image*), 37
Stack() (*in module LineStacker.OneD_Stacker*), 40
stack_estimator() (*in module LineStacker.analysisTools*), 45
subsample_OneD() (*in module LineStacker.analysisTools*), 44

V

velocity_to_frequency() (*in module LineStacker.OneD_Stacker*), 40
velToFreq() (*LineStacker.OneD_Stacker.Spectrum method*), 40
verbose (*in module LineStacker*), 33